TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÀI TẬP LỚN

Học phần: Cấu trúc dữ liệu và Giải thuật

ĐT05: HỆ THỐNG QUẢN LÝ HÀNG TỒN KHO VÀ DOANH THU SIÊU THỊ SỬ DỤNG CÂY CÂN BẰNG AVL

Giảng viên hướng dẫn: **Trần Thị Dung**
Nhóm thực hiện: **Nhóm ĐT05**
Thành viên: TV1 – Đoàn Quốc Nhật – 6651020059 (Nhóm trưởng)
TV2 – Huỳnh Long Nhật – 6651020061
TV3 – Trần Văn Tài – 6651020077
TV4 – Hoàng Xuân Nhuật – 6651020062
TV5 – Nguyễn Duy Khang – 6651020037

TP. Hồ Chí Minh, tháng 11 năm 2026

LỜI NÓI ĐẦU

Trong các hệ thống quản lý bán lẻ, kho hàng là nơi dữ liệu được tra cứu, cập nhật và thống kê liên tục: mỗi lần bán hàng phải kiểm tra tồn kho, mỗi lần nhập hàng phải cập nhật số lượng, và cuối ngày phải tổng hợp doanh thu. Khi số lượng mặt hàng tăng từ vài nghìn lên hàng trăm nghìn, việc chọn cấu trúc dữ liệu phù hợp quyết định trực tiếp tốc độ phản hồi của chương trình. Tìm kiếm tuần tự trên mảng có độ phức tạp $O(n)$ và chậm dần theo quy mô dữ liệu, trong khi cây nhị phân tìm kiếm tự cân bằng AVL luôn giữ chiều cao $O(\log n)$ cho các thao tác thêm, xóa và tìm kiếm.

Bài tập lớn học phần *Cấu trúc dữ liệu và Giải thuật*, đề tài ĐT05, yêu cầu nhóm tự cài đặt cây AVL `AVLTree<K, V>` bằng C++ rồi ứng dụng vào bài toán quản lý hàng tồn kho và doanh thu siêu thị. Chương trình gồm các chức năng: quản lý danh mục mặt hàng (thêm, sửa, xóa, tìm kiếm, tra cứu theo khoảng), nhập và xuất kho có cảnh báo tồn tối thiểu, lập hóa đơn và thống kê doanh thu, lưu và nạp dữ liệu bằng file nhị phân, cùng giao diện dòng lệnh. Nhóm cũng đo thực nghiệm thời gian tìm kiếm của cây AVL so với tìm kiếm tuyến tính ở ba quy mô 1.000, 10.000 và 100.000 mặt hàng.

Báo cáo được chia thành bốn chương:

- **Chương 1** khảo sát nghiệp vụ và phân tích yêu cầu chức năng của hệ thống.
- **Chương 2** trình bày lý thuyết, các phép quay và cài đặt cây AVL, kèm đánh giá độ phức tạp từng thao tác.
- **Chương 3** mô tả cài đặt ba phân hệ: quản lý kho, hóa đơn và doanh thu, file nhị phân và giao diện.
- **Chương 4** trình bày thực nghiệm hiệu năng và phân tích độ phức tạp.

Phần cuối gồm kết luận, tài liệu tham khảo và phụ lục phân công công việc của năm thành viên.

Nhóm xin chân thành cảm ơn giảng viên hướng dẫn đã định hướng và góp ý trong quá trình thực hiện. Do thời gian có hạn, báo cáo khó tránh khỏi thiếu sót; nhóm rất mong nhận được nhận xét của thầy cô để hoàn thiện hơn.

TP. Hồ Chí Minh, tháng 11 năm 2026

Nhóm thực hiện

MỤC LỤC

Lời nói đầu **i**

1 Khảo sát nghiệp vụ và Phân tích bài toán **1**

1.1 Bối cảnh và động cơ lựa chọn đề tài 1

1.2 Phạm vi hệ thống và các tác nhân 1

1.3 Khảo sát quy trình nghiệp vụ 1

1.3.1 Quản lý danh mục và vòng đời mặt hàng 1

1.3.2 Nhập kho và xuất kho 2

1.3.3 Bán hàng và thống kê 2

1.3.4 Lưu và phục hồi 2

1.4 Yêu cầu chức năng 2

1.5 Yêu cầu phi chức năng và ràng buộc thiết kế 3

1.6 Mô hình phân rã chức năng 3

1.7 Luồng xử lý một giao dịch bán hàng 3

1.8 Các trường hợp biên cần kiểm soát 4

1.9 Tiêu chí nghiệm thu tổng thể 4

2 Thiết kế cấu trúc dữ liệu và giải thuật **5**

2.1 Lý do chọn cây AVL 5

2.2 Định nghĩa cây AVL và hệ số cân bằng 5

2.2.1 Chiều cao và hệ số cân bằng 5

2.2.2 Định nghĩa 5

2.2.3 Chiều cao cây AVL 6

2.3 Bốn phép quay cân bằng 6

2.3.1 Trường hợp LL: quay phải 6

2.3.2 Trường hợp RR: quay trái 7

2.3.3 Trường hợp LR: quay trái rồi quay phải 7

2.3.4 Trường hợp RL: quay phải rồi quay trái 7

2.3.5 Tổng hợp cách chọn phép quay 7

2.4 Thêm nút và tái cân bằng 8

2.5 Xóa nút và cân bằng lại 8

2.6 Tìm kiếm và duyệt cây 9

2.6.1 Tìm kiếm theo khóa 9

2.6.2 Duyệt trung thứ tự (LNR) 9

2.7 Tra cứu theo phạm vi (Range Search) 9

2.8 Đánh giá độ phức tạp 10

3 Cài đặt chương trình **11**

3.1 Thiết kế và Cài đặt Module Quản lý Kho hàng . . . 11

3.1.1 Vai trò và ranh giới module . . . 11

3.1.2 Cấu trúc dữ liệu Mặt hàng (`struct MatHang`) . . . 11

3.1.3 Sơ đồ luồng dữ liệu trong module kho . . . 11

3.1.4 Cài đặt các thao tác CRUD . . . 12

3.1.5 Tra cứu theo khoảng mã và khoảng giá . . . 12

3.1.6 Nhập kho và xuất kho có kiểm soát . . . 13

3.1.7 Cảnh báo tồn kho tối thiểu . . . 13

3.1.8 Nạp dữ liệu CSV . . . 13

3.1.9 Bộ kiểm thử nghiệp vụ kho . . . 14

3.1.10 Kết quả chạy kiểm thử tích hợp . . . 14

3.2 Xây dựng Module Quản lý Hóa đơn và Doanh thu . . . 15

3.2.1 Tổng quan và Phân tích Nghiệp vụ . . . 15

3.2.2 Cấu trúc Dữ liệu Cốt lõi (`hoadon.h`) . . . 15

3.2.3 Thiết kế Nguyên mẫu Hàm (Header Modules) . . . 16

3.2.4 Giải thích Giải thuật và Thiết kế Kỹ thuật . . . 18

3.3 Thiết kế và Cài đặt Module Lưu trữ File Nhị phân và Giao diện Console . . . 18

3.3.1 Mục tiêu và ranh giới lưu trữ . . . 18

3.3.2 Đặc tả bố cục file . . . 18

3.3.3 Ghi dữ liệu và thay thế file . . . 19

3.3.4 Kiểm tra khi đọc file . . . 19

3.3.5 Xử lý lỗi và bảo đảm trạng thái nhất quán . . . 20

3.3.6 Mẫu mã kiểm tra header và độ dài file . . . 20

3.3.7 Giao diện Menu Console . . . 20

3.3.8 Xử lý ngoại lệ và vòng đời bộ nhớ . . . 21

3.3.9 Kiểm thử FileIO và các giới hạn . . . 21

3.3.10 Hướng dẫn thao tác demo . . . 21

3.3.11 Đầu ra kiểm thử và cách đọc thông báo . . . 22

4 Thực nghiệm và Đánh giá Hiệu năng **23**

4.1 Mục tiêu thực nghiệm . . . 23

4.2 Môi trường và phương pháp đo . . . 23

4.3 Kết quả đo tìm kiếm . . . 23

4.4 Kết quả đo chèn và xóa . . . 23

4.5 Trực quan hóa kết quả tìm kiếm . . . 24

4.6 Phân tích kết quả và liên hệ độ phức tạp . . . 24

4.7 Giới hạn của phép đo . . . 25

4.8 Trích xuất đầu ra benchmark . . . 25

4.9 Kết luận chương 26

Kết luận **27**

Phụ lục: Phân công công việc và tình trạng xác minh **30**

Phụ lục: Hướng dẫn biên dịch, kiểm thử và nghiệm thu **31**

DANH SÁCH HÌNH VẼ

1.1 Mô hình phân rã chức năng và luồng dữ liệu giữa các phân hệ 3

2.1 Trường hợp LL: quay phải tại z. 6
2.2 Trường hợp RR: quay trái tại z. 7
2.3 Trường hợp LR: quay trái tại y rồi quay phải tại z. 7
2.4 Trường hợp RL: quay phải tại y rồi quay trái tại z. 8

3.1 Luồng xử lý tổng quát của module quản lý kho 12
3.2 Ảnh chụp xterm chạy make test trên Linux/container (không phải máy demo Windows) 14
3.3 Bố cục tuần tự của file kho.dat 18
3.4 Quy trình ghi file qua file tạm 19
3.5 Các bước kiểm tra tính hợp lệ khi nạp file 20

4.1 So sánh thời gian tìm kiếm bằng AVL và vector đã sắp xếp 24

DANH SÁCH BẢNG

1.1 Yêu cầu chức năng và cách kiểm chứng 2
1.2 Các trường hợp biên trong nghiệp vụ 4

2.1 Giới hạn chiều cao của cây AVL so với trường hợp xấu nhất của BST. 6
2.2 Chọn phép quay theo hệ số cân bằng. 8
2.3 Độ phức tạp các thao tác trên cây AVL. 10

3.1 Ý nghĩa các trường của MatHang 11
3.2 Ma trận kiểm thử module kho 14
3.3 Các trường trong FileHeader 19
3.4 Các ca kiểm thử FileIO đã chạy 21

4.1 Trung vị thời gian tìm kiếm 1.000 khóa (9 lần đo) 23
4.2 Trung vị thời gian của 100 thao tác cập nhật (9 lần đo) 24
4.3 Phạm vi phân công và bằng chứng trong gói bàn giao 30
4.4 Các tệp nguồn chính trong dự án tích hợp 31
4.5 Độ phức tạp của các thao tác chính 32
4.6 Checklist các bằng chứng kỹ thuật được ghi nhận 32
4.7 Mẫu ghi nhận tập dượt demo (T52) 35
4.8 Mẫu ghi nhận đọc chéo báo cáo (T53) 35

CHƯƠNG 1

KHẢO SÁT NGHIỆP VỤ VÀ PHÂN TÍCH BÀI TOÁN

1.1 Bối cảnh và động cơ lựa chọn đề tài

Trong hoạt động của một siêu thị, dữ liệu hàng hóa thay đổi liên tục do các hoạt động nhập hàng, bán hàng, điều chỉnh tồn, kiểm kê và cập nhật giá. Một danh mục vài chục nghìn mã hàng có thể phát sinh nhiều yêu cầu tra cứu mỗi ngày. Người vận hành cần tìm đúng mặt hàng theo mã, kiểm tra lượng tồn trước khi bán, nhận biết sản phẩm sắp hết và tổng hợp doanh thu theo các mốc thời gian.

Nếu lưu danh mục trong một mảng tuyến tính, phép tìm kiếm theo mã phải lần lượt so sánh các phần tử cho đến khi gặp kết quả hoặc duyệt hết danh sách. Khi số lượng phần tử tăng, thời gian tra cứu tăng theo quy mô dữ liệu. Cây tìm kiếm nhị phân thông thường có thể đạt hiệu năng tốt khi cây cân đối, nhưng thứ tự chèn bất lợi có thể làm chiều cao cây tăng gần bằng số nút. Vì vậy, đề tài lựa chọn cây AVL - một cây tìm kiếm nhị phân tự cân bằng - để lưu trữ danh mục hàng hóa. Trọng tâm của bài tập lớn không chỉ là cài đặt cấu trúc dữ liệu mà còn là tích hợp cấu trúc đó vào các nghiệp vụ kho, hóa đơn, doanh thu và lưu trữ.

1.2 Phạm vi hệ thống và các tác nhân

Phạm vi triển khai là chương trình C++ chạy trên giao diện dòng lệnh (console), sử dụng cây AVL trong bộ nhớ chính và file nhị phân để lưu trạng thái giữa các lần chạy. Hệ thống chưa kết nối cơ sở dữ liệu tập trung, chưa có tài khoản người dùng, phân quyền hoặc giao diện web. Đây là các giới hạn được xác định trước để tập trung vào mục tiêu học phần là thiết kế và đánh giá cấu trúc dữ liệu.

Các tác nhân chính gồm:

- **Nhân viên kho:** xem danh sách, thêm/sửa/xóa mặt hàng, tra cứu theo mã hoặc khoảng giá, nhập kho, xuất kho và theo dõi cảnh báo tồn tối thiểu.
- **Nhân viên bán hàng:** lập hóa đơn, chọn mặt hàng, nhập số lượng mua, kiểm tra tồn và xác nhận giao dịch.
- **Người quản lý:** xem danh sách hóa đơn, tổng hợp doanh thu theo ngày/tháng và xem nhóm sản phẩm bán chạy.
- **Người vận hành hệ thống:** lưu, khôi phục dữ liệu; kiểm tra lỗi file; chạy bộ kiểm thử và benchmark.

Một người có thể đảm nhiệm nhiều vai trò trong bản demo. Việc tách tác nhân ở đây nhằm làm rõ mục đích sử dụng và quyền thao tác về mặt nghiệp vụ, không hàm ý hệ thống đã có cơ chế xác thực người dùng.

1.3 Khảo sát quy trình nghiệp vụ

1.3.1 Quản lý danh mục và vòng đời mặt hàng

Mỗi mặt hàng được nhận diện bằng mã duy nhất. Khi thêm mới, hệ thống cần kiểm tra mã chưa tồn tại, độ dài mã và tên không vượt quá kích thước mảng ký tự cố định, đơn giá và số lượng không âm. Khi sửa, mã được dùng để

tìm bản ghi hiện có; chương trình cập nhật những thuộc tính được phép thay đổi mà không tạo thêm khóa trùng. Khi xóa, nút tương ứng được loại khỏi cây và các nút tổ tiên được cân bằng lại.

1.3.2 Nhập kho và xuất kho

Nhập kho làm tăng số lượng tồn. Số lượng nhập phải là số nguyên dương; mã hàng phải tồn tại. Xuất kho cần kiểm tra cả mã hàng lẫn lượng tồn. Nếu số lượng yêu cầu lớn hơn số lượng đang có, thao tác bị từ chối và không được thay đổi dữ liệu. Sau khi xuất thành công, chương trình so sánh lượng tồn mới với ngưỡng tối thiểu để hiển thị cảnh báo.

1.3.3 Bán hàng và thống kê

Một hóa đơn gồm mã hóa đơn, ngày lập, danh sách chi tiết và tổng tiền. Mỗi dòng chi tiết chứa mã mặt hàng, số lượng mua và đơn giá tại thời điểm lập hóa đơn. Hệ thống kiểm tra số lượng mua, số dòng chi tiết và khả năng đáp ứng từ kho. Tổng tiền được cộng dồn theo kiểu số nguyên 64-bit để giảm nguy cơ tràn số khi nhân đơn giá với số lượng lớn. Sau giao dịch, số lượng tồn được cập nhật và hóa đơn được đưa vào danh sách để thống kê.

1.3.4 Lưu và phục hồi

Khi thoát hoặc khi người dùng chủ động chọn chức năng lưu, chương trình ghi trạng thái kho và danh sách hóa đơn ra file nhị phân. Ở lần khởi động tiếp theo, hệ thống thử đọc file đã lưu; nếu không có file hợp lệ thì có thể nạp dữ liệu mẫu CSV để phục vụ thực hành. Nếu file lưu bị hỏng, chương trình phải thông báo lỗi và không nạp một phần dữ liệu lỗi vào cấu trúc đang hoạt động.

1.4 Yêu cầu chức năng

Bảng 1.1 tổng hợp các nhóm yêu cầu chức năng và tiêu chí kiểm chứng tương ứng.

Bảng 1.1: Yêu cầu chức năng và cách kiểm chứng

Mã	Nhóm	Yêu cầu	Cách kiểm chứng
FR01	Danh mục	Thêm, sửa, xóa và tìm theo mã; không cho phép mã trùng.	Bộ test AVL và test kho.
FR02	Tra cứu	Tìm theo khoảng mã và lọc theo khoảng giá.	Đối chiếu tập kết quả mẫu.
FR03	Kho	Nhập/xuất số lượng; chặn xuất quá tồn.	Test nhập-xuất và tồn sau giao dịch.
FR04	Cảnh báo	Liệt kê hàng có tồn không vượt ngưỡng tối thiểu.	Test danh sách cảnh báo.
FR05	Hóa đơn	Lập hóa đơn, tính tổng tiền và trừ tồn tự động.	Kiểm tra chi tiết và tồn kho sau bán.
FR06	Doanh thu	Tổng hợp theo ngày/tháng và xếp hạng Top N.	So sánh với phép tính tay trên dữ liệu nhỏ.
FR07	Lưu trữ	Ghi/đọc kho và hóa đơn; từ chối file sai định dạng.	Bộ test FileIO.
FR08	Giao diện	Có menu, kiểm tra lựa chọn và xử lý nhập liệu sai.	Chạy thử các nhánh menu.
FR09	Đánh giá	So sánh AVL với tìm tuyến tính trên các quy mô dữ liệu.	Chạy benchmark có seed cố định.

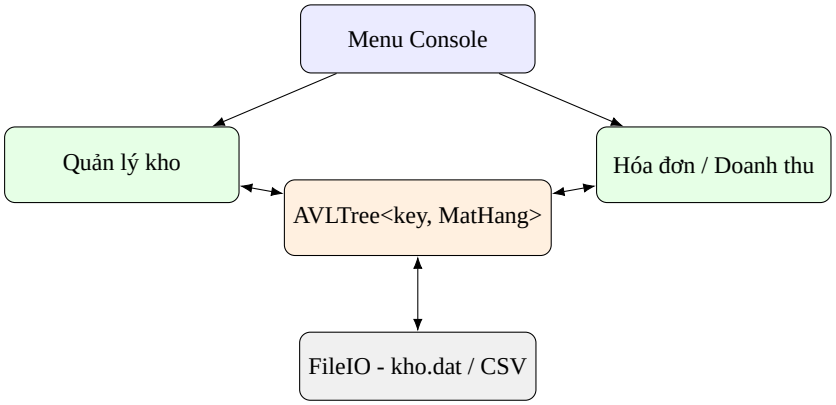
1.5 Yêu cầu phi chức năng và ràng buộc thiết kế

- **Hiệu năng:** tìm kiếm chính xác theo khóa trên cây AVL có độ phức tạp $O(\log n)$ trong trường hợp xấu nhất; range search theo khóa có độ phức tạp $O(\log n + k)$ với k là số kết quả.
- **Tính đúng đắn:** sau thao tác chèn/xóa, thứ tự khóa phải được duy trì và hệ số cân bằng của mọi nút thuộc $\{-1, 0, 1\}$.
- **Tính toàn vẹn:** số lượng, đơn giá và ngưỡng tồn không âm; khóa không rỗng và không trùng; file phải có kích thước khớp header.
- **Khả năng bảo trì:** mã được chia thành header và source theo module; các giao diện hàm được thống nhất trước khi tích hợp.
- **Tính tái lập thực nghiệm:** seed sinh truy vấn được cố định; phương pháp đo và môi trường biên dịch được ghi trong báo cáo.

Các ràng buộc kỹ thuật quan trọng là C++11 [4], khóa cây là `std::string`, giá trị là `MatHang`, mỗi mã/tên mặt hàng được giới hạn theo mảng ký tự cố định và mỗi hóa đơn giới hạn tối đa 20 dòng chi tiết.

1.6 Mô hình phân rã chức năng

Hình 1.1 mô tả cách các phân hệ trao đổi dữ liệu. Cây AVL là cấu trúc trung tâm của danh mục kho; phân hệ hóa đơn tham chiếu đến mã mặt hàng trong cây để xác nhận tồn và cập nhật lượng hàng. Module FileIO nhận dữ liệu đã duyệt từ cây và danh sách hóa đơn để ghi ra file, hoặc kiểm tra dữ liệu đọc được trước khi nạp vào bộ nhớ.



Hình 1.1: Mô hình phân rã chức năng và luồng dữ liệu giữa các phân hệ

1.7 Luồng xử lý một giao dịch bán hàng

Một giao dịch bán hàng cần giữ các bước kiểm tra theo đúng thứ tự để tránh trừ tồn khi dữ liệu chưa hợp lệ. Quy trình khái quát:

1. Người dùng chọn chức năng lập hóa đơn và nhập mã mặt hàng.
2. Hệ thống tìm mã trong cây AVL. Nếu không tìm thấy, dòng hàng không được thêm vào hóa đơn.
3. Người dùng nhập số lượng mua. Chương trình từ chối số lượng không dương hoặc vượt quá lượng tồn.

4. Hệ thống lấy đơn giá, tính thành tiền, kiểm tra giới hạn số dòng và cập nhật chi tiết hóa đơn.
5. Khi xác nhận hóa đơn, lượng tồn được trừ và tổng tiền được lưu vào bản ghi hóa đơn.
6. Người dùng có thể lưu dữ liệu xuống file nhị phân để phục hồi trong phiên làm việc sau.

1.8 Các trường hợp biên cần kiểm soát

Bảng 1.2 liệt kê các tình huống dễ gây lỗi nếu chỉ kiểm tra luồng sử dụng thông thường.

Bảng 1.2: Các trường hợp biên trong nghiệp vụ

Tình huống	**Rủi ro**	**Cách xử lý mong đợi**
Thêm mã đã tồn tại	Ghi đè nhầm mặt hàng	Từ chối và thông báo mã trùng.
Xuất số lượng âm hoặc bằng 0	Dữ liệu tồn kho sai	Từ chối trước khi thay đổi tồn.
Xuất vượt tồn	Phát sinh tồn âm	Giữ nguyên tồn và báo lỗi.
Tìm mã không tồn tại	Truy cập con trỏ rỗng	Kiểm tra nullptr trước khi dùng.
File bị cắt cụt	Đọc thiếu bản ghi	Đối chiếu kích thước và từ chối file.
Nhập lựa chọn dạng chữ	Luồng cin ở trạng thái lỗi	clear và ignore trước khi hỏi lại.
Hóa đơn quá 20 dòng	Vượt giới hạn mảng chi tiết	Từ chối thêm dòng tiếp theo.

1.9 Tiêu chí nghiệm thu tổng thể

Một phân hệ chỉ được xem là kiểm chứng ở mức kỹ thuật khi có mã nguồn, có cách biên dịch và có bằng chứng chạy phù hợp. Đối với cấu trúc AVL, bằng chứng là bộ test phép quay, xóa và range search. Đối với nghiệp vụ kho, bằng chứng là các ca CRUD, nhập/xuất, cảnh báo và nạp dữ liệu mẫu. Đối với FileIO, bằng chứng là các ca ghi/đọc và các ca từ chối file lỗi. Đối với benchmark, cần ghi rõ số truy vấn, số lần lặp, thống kê được sử dụng và môi trường thực thi.

Các thao tác mang tính quy trình như tạo repository, mở/merge pull request, cập nhật Jira, chia sẻ tài liệu cho giảng viên, đọc chéo và tập dượt demo không thể suy ra chỉ từ việc có file trong gói nộp. Những việc này cần bằng chứng tương ứng từ nền tảng hoặc biên bản nhóm; báo cáo không gắn trạng thái hoàn thành nếu chưa có bằng chứng đó.

CHƯƠNG 2

THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Chương này trình bày cơ sở lý thuyết và thiết kế của cây cân bằng AVL được chọn làm cấu trúc dữ liệu chính cho hệ thống quản lý hàng tồn kho. Nội dung gồm lý do chọn cấu trúc (mục 2.1), định nghĩa và hệ số cân bằng (mục 2.2), bốn phép quay (mục 2.3), các thao tác thêm, xóa, tìm kiếm, duyệt và Range Search (mục 2.4 đến mục 2.7) và đánh giá độ phức tạp (mục 2.8).

2.1 Lý do chọn cây AVL

Danh mục mặt hàng của siêu thị thay đổi liên tục: mặt hàng mới được thêm vào, mặt hàng ngừng kinh doanh bị xóa, số lượng tồn kho thay đổi sau mỗi lần nhập và xuất. Đồng thời hệ thống phải tra cứu một mặt hàng theo mã gần như tức thì và liệt kê các mặt hàng trong một khoảng mã. Các yêu cầu đó dẫn đến ba điều kiện đối với cấu trúc dữ liệu: tìm, thêm và xóa nhanh; duyệt được theo thứ tự khóa; và tra cứu được theo khoảng.

Cây nhị phân tìm kiếm (BST) thông thường đáp ứng điều kiện thứ hai và thứ ba, nhưng độ phức tạp của nó phụ thuộc vào hình dạng cây. Nếu các mã hàng được nhập theo thứ tự tăng dần, BST suy biến thành một danh sách liên kết có chiều cao n và mỗi thao tác tốn $O(n)$. Cây AVL giữ chiều cao luôn ở mức $O(\log n)$ bất kể thứ tự nhập [2, 1], nên được chọn cho đề tài này.

2.2 Định nghĩa cây AVL và hệ số cân bằng

2.2.1 Chiều cao và hệ số cân bằng

Với mỗi nút v của cây, ký hiệu $h(v)$ là chiều cao của cây con gốc v. Quy ước dùng trong cài đặt: cây rỗng có chiều cao 0, nút lá có chiều cao 1, và

$$h(v) = 1 + \max\big(h(\text{con trái}),\, h(\text{con phải})\big).$$

Hệ số cân bằng (balance factor) của nút v là hiệu chiều cao hai cây con:

$$BF(v) = h(\text{con trái}) - h(\text{con phải}).$$

2.2.2 Định nghĩa

Cây AVL là cây nhị phân tìm kiếm trong đó mọi nút v đều thỏa $BF(v) \in \{-1, 0, 1\}$. Nút có $BF > 1$ gọi là lệch trái, nút có $BF < -1$ gọi là lệch phải. Trong chương trình, mỗi nút `AVLNode` lưu sẵn trường `height`, nên `getBalanceFactor` chỉ cần lấy hiệu hai trường này mà không phải duyệt lại cây con.

2.2.3 Chiều cao cây AVL

Gọi $N(h)$ là số nút ít nhất của một cây AVL có chiều cao h. Cây ít nút nhất có một cây con cao $h-1$ và cây con kia cao $h-2$ (lệch tối đa mà vẫn hợp lệ), do đó

$$N(1) = 1, \quad N(2) = 2, \quad N(h) = N(h-1) + N(h-2) + 1.$$

Quy hồi này cùng dạng với dãy Fibonacci $F_1 = F_2 = 1,\ F_k = F_{k-1} + F_{k-2}$ [3]: bằng quy nạp, $N(h) = F_{h+2} - 1$. Vì $F_{h+2} \geq \varphi^h$ với $\varphi = \frac{1+\sqrt{5}}{2} \approx 1{,}618$, một cây AVL có n nút và chiều cao h thỏa $n \geq \varphi^h - 1$, suy ra

$$h \ \leq \ \log_{\varphi}(n+1) \ \approx \ 1{,}44 \ \log_2(n+1).$$

Như vậy chiều cao cây AVL luôn là $O(\log n)$, và mọi thao tác đi dọc một đường từ gốc xuống lá đều có độ phức tạp $O(\log n)$. Bảng 2.1 cho thấy giá trị này với các kích thước cây điển hình.

Số nút n	Chiều cao tối đa của AVL ($\leq 1{,}44\log_2(n+1)$)	Chiều cao tối đa của BST
1 000	14	1 000
10 000	19	10 000
100 000	23	100 000

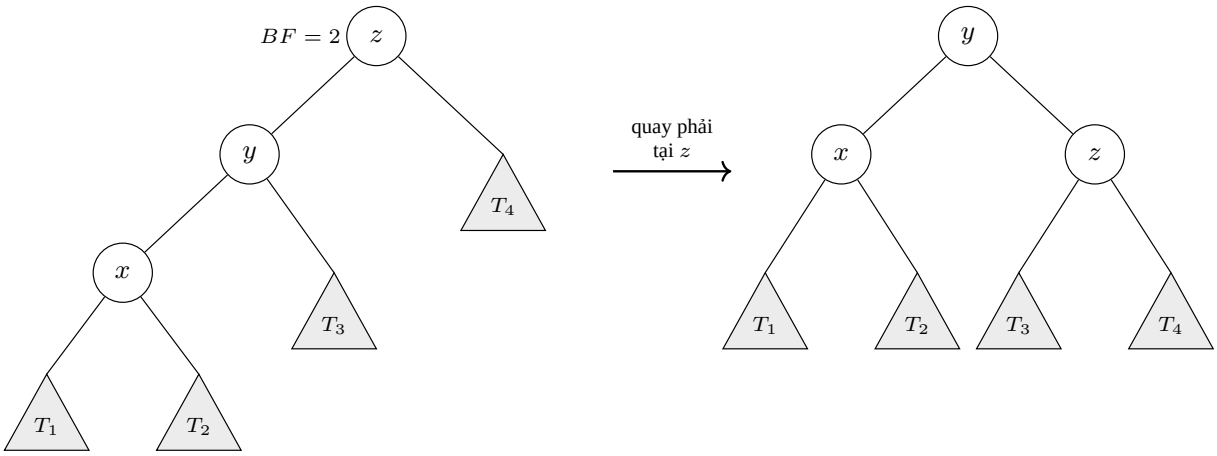
Bảng 2.1: Giới hạn chiều cao của cây AVL so với trường hợp xấu nhất của BST.

2.3 Bốn phép quay cân bằng

Sau khi thêm hoặc xóa một nút, một số nút trên đường đi từ nút đó lên gốc có thể có $BF = \pm 2$. Gọi z là nút lệch *thấp nhất* (gần lá nhất), y là con của z nằm ở phía cao hơn và x là con của y nằm ở phía cao hơn (hoặc bằng nhau khi xóa). Tùy vị trí của y và x mà ta có bốn trường hợp. Trong các hình dưới đây, T_1, T_2, T_3, T_4 là các cây con (vẽ bằng tam giác) và thứ tự khóa từ trái sang phải được giữ nguyên trước và sau khi quay.

2.3.1 Trường hợp LL: quay phải

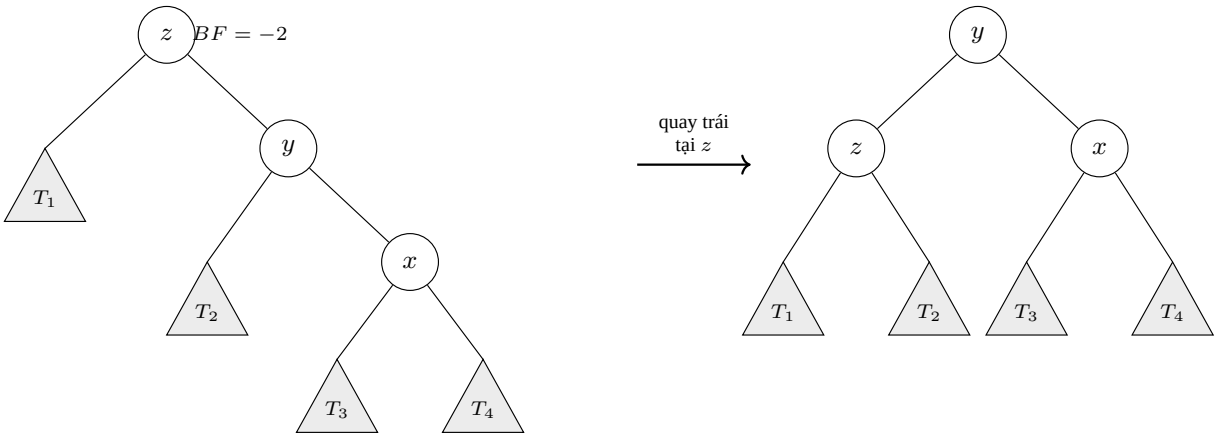
Nút z lệch trái ($BF(z) = 2$) và con trái y cũng lệch trái hoặc cân bằng ($BF(y) \geq 0$). Ta quay phải tại z: y trở thành gốc của cây con, z trở thành con phải của y, cây con phải cũ của y (tức T_3) được chuyển thành con trái của z.



Hình 2.1: Trường hợp LL: quay phải tại z.

2.3.2 Trường hợp RR: quay trái

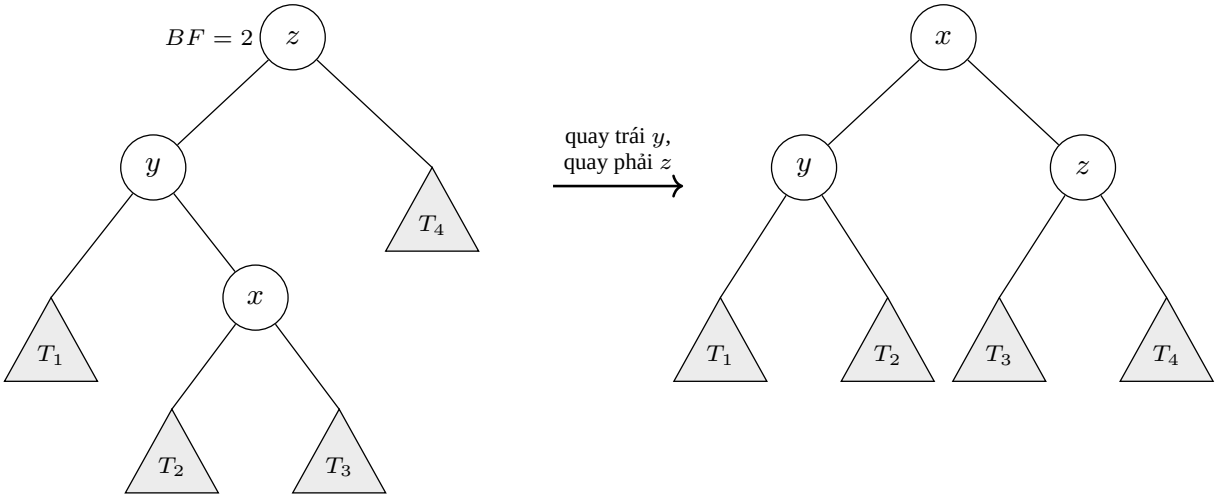
Đối xứng với LL: z lệch phải ($BF(z) = -2$) và con phải y lệch phải hoặc cân bằng ($BF(y) \leq 0$). Ta quay trái tại z.



Hình 2.2: Trường hợp RR: quay trái tại z.

2.3.3 Trường hợp LR: quay trái rồi quay phải

Nút z lệch trái nhưng con trái y lệch phải ($BF(y) < 0$), nên một phép quay phải đơn tại z chưa đủ. Ta quay trái tại y để đưa về trường hợp LL, sau đó quay phải tại z. Kết quả là x (cháu) trở thành gốc cây con.



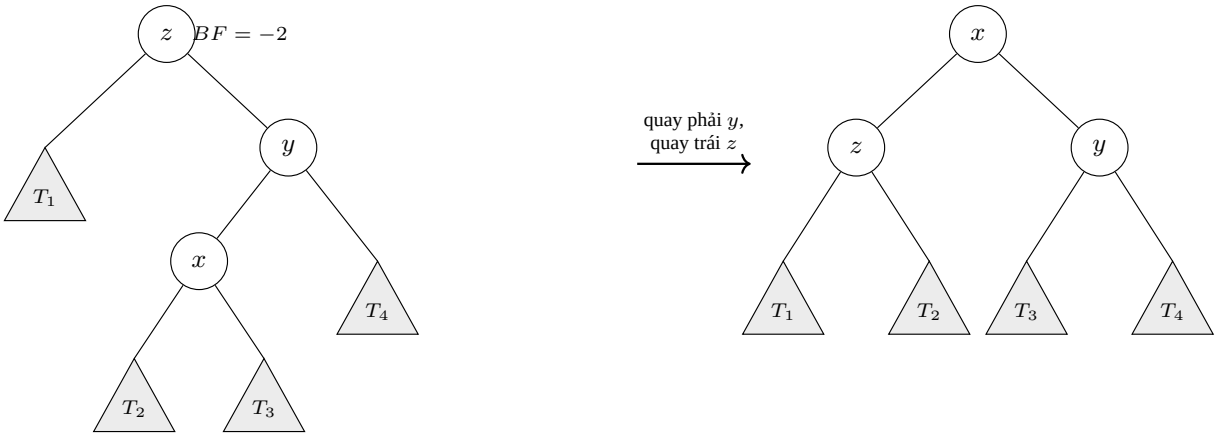
Hình 2.3: Trường hợp LR: quay trái tại y rồi quay phải tại z.

2.3.4 Trường hợp RL: quay phải rồi quay trái

Đối xứng với LR: z lệch phải nhưng con phải y lệch trái ($BF(y) > 0$). Ta quay phải tại y rồi quay trái tại z.

2.3.5 Tổng hợp cách chọn phép quay

Phép quay được chọn theo BF của nút lệch z và của con y ở phía lệch (bảng 2.2). Trong chương trình, hàm `rotateRight` và `rotateLeft` cài hai phép quay đơn; các trường hợp LR và RL được ghép từ hai phép quay đơn. Sau mỗi phép quay, chiều cao của hai nút bị đổi chỗ được tính lại theo thứ tự từ nút thấp lên nút cao.



Hình 2.4: Trường hợp RL: quay phải tại y rồi quay trái tại z.

$BF(z)$	$BF(y)$ của con lệch	Trường hợp	Phép quay
≥ 2	≥ 0	LL	quay phải tại z
≥ 2	< 0	LR	quay trái tại y, quay phải tại z
≤ -2	≤ 0	RR	quay trái tại z
≤ -2	> 0	RL	quay phải tại y, quay trái tại z

Bảng 2.2: Chọn phép quay theo hệ số cân bằng.

Mỗi phép quay chỉ đổi một số hữu hạn con trỏ nên tốn $O(1)$, và không làm đổi thứ tự duyệt *inorder* của các khóa: cây sau khi quay vẫn là cây nhị phân tìm kiếm hợp lệ.

2.4 Thêm nút và tái cân bằng

Thao tác thêm (`insert`) gồm hai pha, thực hiện trong cùng một hàm đệ quy `insertNode`:

1. **Đi xuống:** so sánh khóa mới với khóa của nút hiện tại, đi sang trái nếu nhỏ hơn và sang phải nếu lớn hơn, cho đến khi gặp vị trí rỗng thì tạo nút lá mới. Nếu gặp khóa trùng, thao tác dừng và trả về `false`, cây không thay đổi.
2. **Đi lên:** trên đường quay lại gốc, mỗi nút cập nhật lại `height`, tính BF, và nếu $|BF| = 2$ thì chọn một trong bốn phép quay ở bảng 2.2.

Sau khi thêm một nút, chiều cao của cây con bị lệch quay về đúng giá trị trước khi thêm, nên *nhiều nhất một lần tái cân bằng* (một phép quay đơn hoặc một cặp phép quay kép) là đủ để khôi phục tính cân bằng cho toàn bộ cây. Vì đường đi có độ dài $O(\log n)$ và mỗi phép quay tốn $O(1)$, thao tác thêm tốn $O(\log n)$.

2.5 Xóa nút và cân bằng lại

Thao tác xóa (`remove`) tìm nút theo khóa rồi xử lý ba trường hợp:

1. **Nút lá:** xóa nút, con trỏ của cha trở thành rỗng.
2. **Nút có một con:** xóa nút và nối con duy nhất của nó vào cha.
3. **Nút có hai con:** tìm nút nhỏ nhất của cây con phải (nút kế tiếp theo thứ tự khóa, hàm `getMinValueNode`), chép khóa và dữ liệu của nó vào nút cần xóa, rồi xóa nút kế tiếp khỏi cây con phải. Nút kế tiếp không có con trái nên rơi vào trường hợp 1 hoặc 2.

Trên đường quay lại gốc, mỗi nút cập nhật `height` và tái cân bằng giống thao tác thêm. Có hai điểm khác so với thao tác thêm.

- Sau khi xóa, con y của nút lệch z có thể có $BF(y) = 0$. Khi đó phải dùng phép quay *đơn* (LL hoặc RR), vì quay kép sẽ không cân bằng lại cây. Điều kiện chọn quay kép chỉ là $BF(y)$ ngược chiều với $BF(z)$ (bảng 2.2).
- Một phép quay có thể làm chiều cao cây con giảm đi 1, khiến nút tổ tiên phía trên tiếp tục bị lệch. Vì vậy xóa có thể cần tái cân bằng ở *nhiều* nút trên đường đi, tối đa $O(\log n)$ phép quay.

Tổng độ phức tạp của thao tác xóa vẫn là $O(\log n)$: đi xuống tìm nút, tìm nút kế tiếp và quay lại đều dọc theo một đường từ gốc xuống lá.

2.6 Tìm kiếm và duyệt cây

2.6.1 Tìm kiếm theo khóa

Hàm `search` dùng tính chất của cây nhị phân tìm kiếm: so sánh khóa cần tìm với khóa của nút hiện tại, đi sang trái nếu nhỏ hơn, sang phải nếu lớn hơn, và dừng khi bằng hoặc khi hết cây. Số bước không vượt quá chiều cao cây, tức $O(\log n)$ nhờ tính chất ở mục 2.2.3. Hàm trả về con trỏ tới dữ liệu thực tế của nút (hoặc `nullptr` nếu không có khóa), nên có thể cập nhật trực tiếp, chẳng hạn số lượng tồn kho sau một lần nhập hoặc xuất, mà không phải xóa rồi thêm lại.

2.6.2 Duyệt trung thứ tự (LNR)

Duyệt *inorder* thăm cây con trái, rồi nút hiện tại, rồi cây con phải. Do tính chất của cây nhị phân tìm kiếm, kết quả là danh sách dữ liệu theo khóa tăng dần. Hàm `inorder` dùng thao tác này để lập danh sách mặt hàng theo mã tăng dần cho các chức năng liệt kê và báo cáo, đồng thời là cách duy nhất cần dùng khi ghi toàn bộ cây ra tệp theo thứ tự khóa. Mỗi nút được thăm đúng một lần nên độ phức tạp là $O(n)$.

2.7 Tra cứu theo phạm vi (Range Search)

Range Search trả về tất cả phần tử có khóa nằm trong đoạn $[lo, hi]$, theo thứ tự tăng dần. Ví dụ: liệt kê các mặt hàng có mã từ `SP010` đến `SP050`. Thay vì duyệt cả cây rồi lọc, thuật toán chỉ đi vào những nhánh có thể chứa kết quả (hàm `rangeSearchNode`). Tại nút v với khóa k:

- nếu $lo < k$, cây con trái có thể chứa khóa trong đoạn, nên đi sang trái;
- nếu $lo \le k \le hi$, ghi nhận nút v vào kết quả;
- nếu $k < hi$, cây con phải có thể chứa khóa trong đoạn, nên đi sang phải.

Thứ tự *trái, nút, phải* bảo đảm kết quả đã được sắp tăng dần. Nhánh nào nằm hoàn toàn ngoài đoạn bị bỏ qua ngay. Các nút được thăm chia thành hai loại: các nút nằm trên hai đường từ gốc đến lo và đến hi, có $O(\log n)$ nút, và các nút nằm trong kết quả, có k nút. Do đó thời gian là $O(\log n + k)$, thấp hơn nhiều so với $O(n)$ khi duyệt toàn bộ nếu đoạn truy vấn nhỏ. Khi $lo > hi$ khoảng rỗng và không có phần tử nào được trả về.

Cây được tổ chức theo *khóa*, nên Range Search theo khoảng mã hàng dùng trực tiếp khóa chính. Để tra cứu theo khoảng giá, nhiều mặt hàng có thể cùng giá trong khi cây AVL không cho phép trùng khóa. Cách xử lý là

dùng khóa ghép $(\text{giá}, \text{mã hàng})$ với phép so sánh giá trước, mã hàng sau, và dùng cận dưới $(g_{\min}, \text{rỗng})$ và cận trên $(g_{\max}, \text{ký tự lớn nhất})$ cho lo và hi.

2.8 Đánh giá độ phức tạp

Bảng 2.3 tổng hợp độ phức tạp thời gian của các thao tác, với n là số nút của cây và k là số kết quả của Range Search. Mọi giá trị đều suy ra từ chiều cao $h = O(\log n)$ ở mục 2.2.3.

Thao tác	Trung bình	Xấu nhất	Ghi chú
Thêm (`insert`)	$O(\log n)$	$O(\log n)$	tối đa 1 lần tái cân bằng
Xóa (`remove`)	$O(\log n)$	$O(\log n)$	có thể quay ở nhiều nút
Tìm (`search`)	$O(\log n)$	$O(\log n)$	
Duyệt (`inorder`)	$O(n)$	$O(n)$	mỗi nút thăm một lần
Range Search	$O(\log n + k)$	$O(\log n + k)$	k là số kết quả
Bộ nhớ	$O(n)$	$O(n)$	thêm 1 trường `height` mỗi nút

Bảng 2.3: Độ phức tạp các thao tác trên cây AVL.

Với BST thông thường, các thao tác thêm, xóa và tìm có độ phức tạp trung bình $O(\log n)$ nhưng xấu nhất là $O(n)$ khi dữ liệu nhập theo thứ tự. Cây AVL đánh đổi một chút chi phí cho việc quay và một trường `height` trên mỗi nút để có bảo đảm $O(\log n)$ trong *mọi* trường hợp, điều cần thiết khi mã hàng thường được cấp theo thứ tự tăng dần.

Kết quả đo thực tế trên bộ test (chương 4) xác nhận các giá trị này: với 10^3, 10^4 và 10^5 khóa ngẫu nhiên, chiều cao đo được lần lượt là 12, 16 và 20, đều thấp hơn giới hạn lý thuyết $1{,}44 \log_2(n+1)$ ở bảng 2.1.

CHƯƠNG 3

CÀI ĐẶT CHƯƠNG TRÌNH

3.1 Thiết kế và Cài đặt Module Quản lý Kho hàng

3.1.1 Vai trò và ranh giới module

Module quản lý kho chịu trách nhiệm thao tác trên danh mục mặt hàng và các nghiệp vụ làm thay đổi số lượng tồn. Module không tự cài đặt cây tìm kiếm; thay vào đó, module dùng bí danh `KhoHang = AVLTree<std::string, MatHang>` và gọi các thao tác công khai của cây. Cách tách này giúp phần nghiệp vụ không phụ thuộc vào cách cài đặt nút, phép quay hoặc quản lý bộ nhớ ở bên trong AVLTree.

3.1.2 Cấu trúc dữ liệu Mặt hàng (`struct MatHang`)

Cấu trúc `MatHang` (định nghĩa trong `include/mathang.h`) lưu các thuộc tính cần thiết cho tra cứu và tính tồn kho. Các mảng ký tự có kích thước cố định giúp bản ghi dễ ghi/đọc theo khối trong bản cài đặt FileIO hiện tại. Khi gán chuỗi đầu vào, chương trình phải giới hạn độ dài để bảo đảm chuỗi luôn có ký tự kết thúc.

```
struct MatHang {
    char maHang[16];       // Khoa chinh, toi da 15 ky tu
    char tenHang[64];      // Ten, toi da 63 ky tu
    long long donGia;      // Don gia (VND)
    int soLuongTon;        // So luong hien tai
    int tonToiThieu;       // Nguong canh bao
    Ngay ngayNhap;         // Ngay nhap gan nhat
};
```

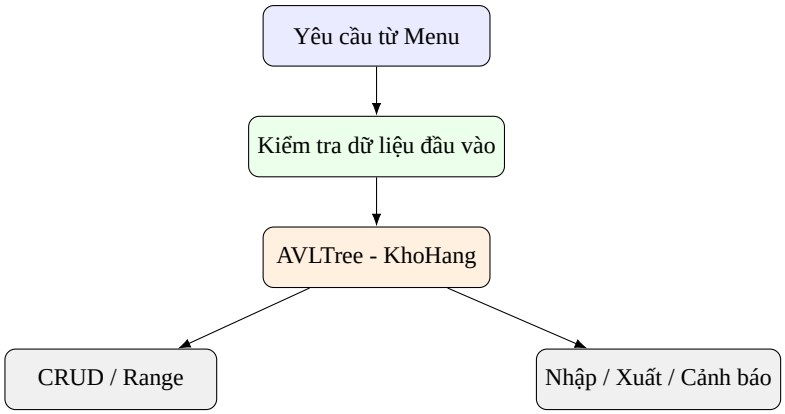
Listing 3.1: Cấu trúc dữ liệu mặt hàng (rút gọn theo định nghĩa dự án)

Bảng 3.1: Ý nghĩa các trường của MatHang

Trường	Kiểu	Ý nghĩa và ràng buộc
`maHang`	`char[16]`	Khóa duy nhất, không rỗng, tối đa 15 ký tự.
`tenHang`	`char[64]`	Tên hiển thị, tối đa 63 ký tự.
`donGia`	`long long`	Đơn giá tính bằng đồng, không âm.
`soLuongTon`	`int`	Lượng hàng trong kho, không âm.
`tonToiThieu`	`int`	Ngưỡng phát cảnh báo, không âm.
`ngayNhap`	`Ngay`	Ngày nhập gần nhất theo cấu trúc ngày/tháng/năm.

3.1.3 Sơ đồ luồng dữ liệu trong module kho

Hình 3.1 thể hiện đường đi từ yêu cầu người dùng đến cấu trúc dữ liệu. Các thao tác theo mã đi qua AVLTree; thao tác cảnh báo duyệt danh sách theo thứ tự; thao tác nhập dữ liệu mẫu đọc CSV rồi lần lượt chèn các mặt hàng vào cây.



Hình 3.1: Luồng xử lý tổng quát của module quản lý kho

3.1.4 Cài đặt các thao tác CRUD

Các thao tác thêm, sửa, xóa và tìm kiếm sử dụng mã mặt hàng làm khóa. Hàm thêm cần từ chối mã trùng trước khi chèn. Hàm sửa tìm con trỏ đến giá trị rồi cập nhật các trường được phép; khóa không thay đổi nên không làm phá vỡ thứ tự của cây. Hàm xóa ủy quyền cho `remove`, nơi cây thực hiện xử lý nút có 0, 1 hoặc 2 con và cân bằng lại đường đi từ nút bị xóa lên gốc.

Ví dụ đoạn mã rút gọn cho thao tác thêm và tìm:

```
bool themMatHang(KhoHang& kho, const MatHang& mh) {
    if (mh.maHang[0] == '\0' || mh.soLuongTon < 0 ||
        mh.donGia < 0 || mh.tonToiThieu < 0) {
        return false;
    }
    return kho.insert(std::string(mh.maHang), mh);
}

MatHang* timMatHang(KhoHang& kho, const std::string& ma) {
    return kho.search(ma);
}
```

Listing 3.2: Minh họa thao tác thêm và tìm theo mã

Đoạn mã trên tập trung vào điều kiện nghiệp vụ và cách ủy quyền. Việc cân bằng, cấp phát nút và xử lý liên kết con thuộc trách nhiệm của AVLTree, không lặp lại trong module kho.

3.1.5 Tra cứu theo khoảng mã và khoảng giá

Tra cứu theo khoảng mã tận dụng thứ tự khóa của cây. Thuật toán so sánh khóa hiện tại với hai biên để bỏ qua những nhánh chắc chắn không thể chứa kết quả. Nếu có k phần tử nằm trong khoảng truy vấn, chi phí kỳ vọng theo cấu trúc AVL là $O(\log n + k)$. Danh sách kết quả được xuất theo thứ tự tăng dần của mã.

Tra cứu theo khoảng giá có đặc điểm khác: cây được tổ chức theo mã hàng, không theo đơn giá. Vì vậy, module duyệt danh sách mặt hàng theo thứ tự trung thứ tự, kiểm tra điều kiện $giaTu \leq donGia \leq giaDen$ cho từng phần tử. Độ phức tạp là $O(n)$; không thể tuyên bố $O(\log n)$ nếu chưa xây dựng thêm chỉ mục theo giá. Khi cần tăng tốc truy vấn giá, có thể bổ sung một cây AVL thứ hai với khóa là cặp (đơn giá, mã hàng), nhưng khi đó mọi thao tác cập nhật mặt hàng phải đồng bộ cả hai chỉ mục.

3.1.6 Nhập kho và xuất kho có kiểm soát

Với nhập kho, điều kiện hợp lệ gồm mã hàng tồn tại, số lượng nhập lớn hơn 0 và ngày nhập hợp lệ. Trước khi cộng số lượng nhập vào tồn kho, chương trình kiểm tra tồn hiện tại không âm và tổng mới không vượt giới hạn của kiểu `int`. Cụ thể, thay vì thực hiện phép cộng rồi mới kiểm tra kết quả, chương trình so sánh `soLuongNhap` với `numeric_limits<int>::max() - soLuongTon`. Nếu lượng nhập lớn hơn phần dung lượng còn lại, thao tác bị từ chối và số lượng tồn được giữ nguyên. Cách kiểm tra trước phép cộng tránh tràn số nguyên có dấu, vốn có thể tạo ra giá trị sai và làm sai lệch dữ liệu kho. Với xuất kho, chương trình kiểm tra lượng xuất dương và không vượt tồn trước khi trừ. Trình tự kiểm tra quan trọng vì nếu cập nhật trước rồi mới phát hiện điều kiện sai, dữ liệu có thể bị thay đổi một phần. Ngày nhập được kiểm tra theo lịch thực tế bằng hàm dùng chung `ngayHopLe()` trong `ngay.h`: tháng 4, 6, 9, 11 chỉ có 30 ngày, tháng 2 có 28 hoặc 29 ngày tùy năm nhuận (chia hết cho 400, hoặc chia hết cho 4 mà không chia hết cho 100). Vì vậy 31/02/2026, 31/04/2026 và 29/02/2027 bị từ chối, còn 29/02/2028 được chấp nhận. Các hàm `themMatHang`, `suaMatHang` và `nhapKho` đều kiểm tra ngày trước khi thay đổi bất kỳ trường nào, nên một thao tác bị từ chối không để lại dữ liệu sửa dở dang.

```
bool xuatKho(KhoHang& kho, const string& maHang,
             int soLuongXuat) {
    if (soLuongXuat <= 0) return false;
    MatHang* mh = kho.search(maHang);
    if (mh == nullptr || soLuongXuat > mh->soLuongTon)
        return false;
    mh->soLuongTon -= soLuongXuat;
    return true;
}
```

Listing 3.3: Đoạn kiểm tra xuất kho, trích theo logic module

Sau khi xuất thành công, nếu số lượng còn lại nhỏ hơn hoặc bằng ngưỡng tồn tối thiểu, giao diện phát thông báo để nhân viên biết cần xem xét nhập bổ sung. Thông báo không tự động tạo đơn mua hàng, bởi quyết định đặt hàng thuộc quy trình quản lý và chưa nằm trong phạm vi đề tài.

3.1.7 Cảnh báo tồn kho tối thiểu

Hàm `layDanhSachCanhBaoTonKho` duyệt danh sách lấy từ `kho.inorder()`, sau đó lấy các mặt hàng thỏa điều kiện $soLuongTon \leq tonToiThieu$. Lượng nhập gợi ý trong phiên bản hiện tại được tính theo công thức:

$$goiYNhap = \max(0, 2 \times tonToiThieu - soLuongTon).$$

Công thức này đặt mục tiêu sau khi nhập, lượng hàng đạt xấp xỉ hai lần ngưỡng tối thiểu. Đây chỉ là quy tắc gợi ý phục vụ minh họa; hệ thống thực tế cần xét tốc độ bán, thời gian giao hàng, lượng đặt tối thiểu và tính mùa vụ.

3.1.8 Nạp dữ liệu CSV

Dữ liệu mẫu được lưu tại `data/du_lieu_mau.csv`. Luồng nạp đọc từng dòng, tách các trường theo dấu phẩy, tạo đối tượng mặt hàng rồi chèn vào cây. Dữ liệu mẫu được dùng cho kiểm thử và demo, không phải dữ liệu giao dịch của một siêu thị thật. Trong kho mã nguồn đã loại bỏ bản sao `mau_mathang.csv`; chương trình dùng một tên file chuẩn để tránh hai bản dữ liệu trôi lệch nhau.

3.1.9 Bộ kiểm thử nghiệp vụ kho

Các ca kiểm thử tích hợp trong `tests/test_kho.cpp` kiểm tra các nhóm: nạp 50 mặt hàng; thêm mới và từ chối mã trùng; sửa thông tin; xóa và xác nhận không còn tìm thấy; nhập kho; từ chối xuất vượt tồn; xuất hợp lệ; cảnh báo tồn tối thiểu; range search theo mã/giá; lưu và nạp kho cùng hóa đơn qua FileIO. Việc kiểm thử trên AVLTree thật giúp xác nhận module kho không chỉ chạy với một cấu trúc giả lập.

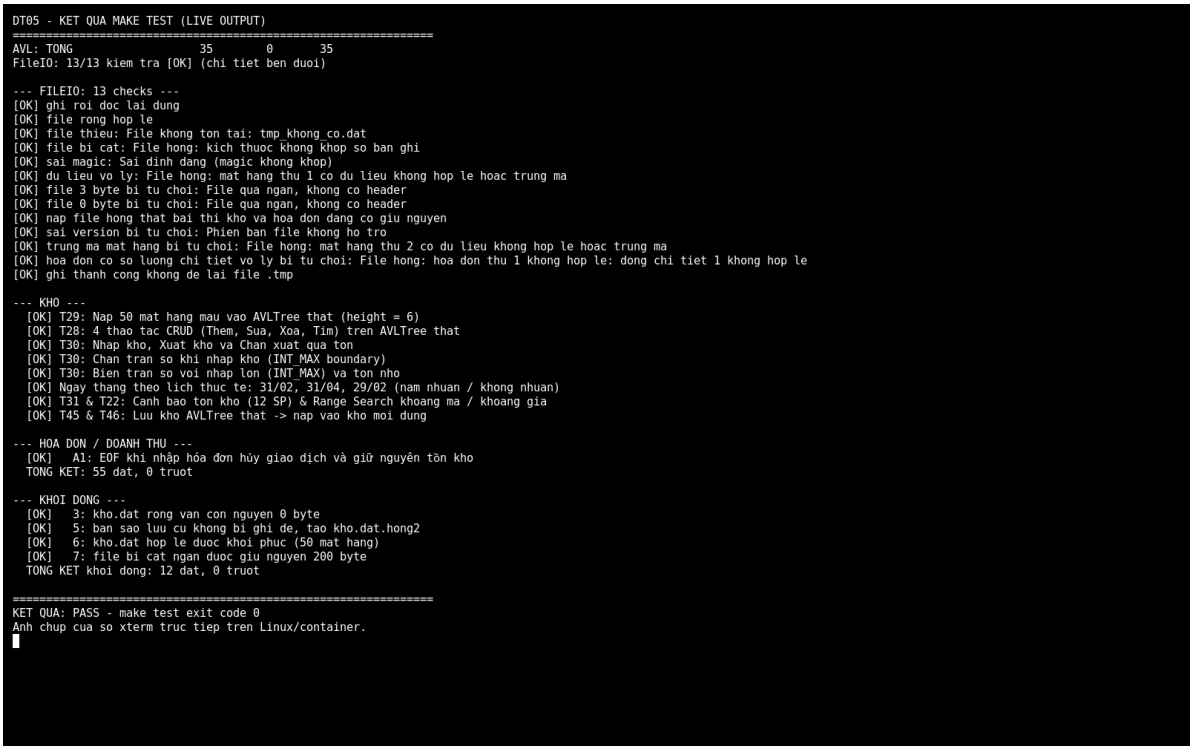
Bảng 3.2: Ma trận kiểm thử module kho

Mã	Ca kiểm thử	Kết quả mong đợi	Bằng chứng
K01	Nạp CSV	Nạp đủ 50 mã; cây có 50 nút.	Log test kho.
K02	Thêm mã mới	Trả về thành công và tăng kích thước cây.	Assertion T28.
K03	Thêm mã trùng	Trả về thất bại, không tăng kích thước.	Assertion T28.
K04	Sửa thông tin	Tên/giá/ngưỡng được cập nhật.	Assertion T28.
K05	Xóa mặt hàng	Mã bị xóa không còn tìm thấy.	Assertion T28.
K06	Xuất vượt tồn	Từ chối và giữ nguyên số lượng tồn.	Assertion T30.
K07	Cảnh báo	Danh sách cảnh báo không rỗng với dữ liệu thử.	Assertion T31.
K08	Range search	Trả đúng 5 mã trong MH001..MH005.	Assertion T22.

Kết quả kiểm thử là bằng chứng cho các đường đi được mô tả, không thay thế cho kiểm thử giao diện người dùng ở mọi tổ hợp nhập liệu. Trước khi bảo vệ, nhóm nên trình diễn trực tiếp ít nhất một trường hợp hợp lệ và một trường hợp bị từ chối để thể hiện rõ quy tắc nghiệp vụ.

3.1.10 Kết quả chạy kiểm thử tích hợp

Hình 3.2 là ảnh chụp cửa sổ xterm trong lúc chạy `make test` trên Linux/container. Các dòng báo lỗi trùng mã hoặc xuất vượt tồn là kết quả được mong đợi của ca kiểm thử âm; phần tổng kết cuối cửa sổ được tính từ log của chính lần chạy này. Ảnh không phải ảnh chụp từ máy Windows demo.



Hình 3.2: Ảnh chụp xterm chạy make test trên Linux/container (không phải máy demo Windows)

3.2 Xây dựng Module Quản lý Hóa đơn và Doanh thu

3.2.1 Tổng quan và Phân tích Nghiệp vụ

Trong hệ thống quản lý siêu thị, module Hóa đơn và Doanh thu đóng vai trò ghi nhận các giao dịch bán hàng thực tế, quản lý danh sách chi tiết các mặt hàng trong từng hóa đơn, đồng thời kết nối trực tiếp với Cây cân bằng AVL (`AVLTree`) nhằm tối ưu hóa hiệu năng tra cứu sản phẩm và cập nhật tồn kho. Các chức năng chính bao gồm:

1. **Lập hóa đơn và Trừ tồn kho tự động:** Tra cứu mã hàng trên Cây AVL với độ phức tạp $O(\log n)$, kiểm tra điều kiện tồn kho, tính toán tổng tiền và tự động cập nhật giảm số lượng tồn trên cây theo thời gian thực.
2. **Thống kê Doanh thu:** Tổng hợp doanh thu theo các tiêu chí thời gian (ngày, tháng) phục vụ công tác báo cáo tài chính.
3. **Thống kê Top Mặt hàng Bán chạy:** Xây dựng thuật toán gom nhóm bằng `std::map` và sắp xếp giảm dần bằng `std::sort` để xếp hạng các mặt hàng có doanh số và số lượng tiêu thụ cao nhất.

3.2.2 Cấu trúc Dữ liệu Cốt lõi (hoadon.h)

Cấu trúc dữ liệu được thiết kế đồng bộ, sử dụng mảng tĩnh giới hạn tối đa 20 dòng chi tiết trong mỗi hóa đơn và kiểu dữ liệu `long long` cho các giá trị tài chính nhằm ngăn chặn hiện tượng tràn số, đồng thời tích hợp constructor khởi tạo an toàn bằng `std::memset`:

```
#ifndef HOADON_H
#define HOADON_H

#include <cstring>
#include "ngay.h"

const int MAX_CHI_TIET = 20;

struct ChiTietHoaDon {
    char maHang[16];
    int soLuongMua;
    long long donGia;

    ChiTietHoaDon() : soLuongMua(0), donGia(0) {
        std::memset(maHang, 0, sizeof(maHang));
    }
};

struct HoaDon {
    char maHoaDon[16];
    Ngay ngayLap;
    ChiTietHoaDon danhSachChiTiet[MAX_CHI_TIET];
    int soLuongChiTiet;
    long long tongTien;

    HoaDon() : ngayLap(), soLuongChiTiet(0), tongTien(0) {
        std::memset(maHoaDon, 0, sizeof(maHoaDon));
    }
```

```
};

#endif
```

Listing 3.4: Cấu trúc Dữ liệu Hóa đơn và Chi tiết (hoadon.h)

3.2.3 Thiết kế Nguyên mẫu Hàm (Header Modules)

Phân hệ Lập Hóa đơn và Tương tác Cây AVL (`lap_hoa_don.h`)

```
#ifndef LAP_HOA_DON_H
#define LAP_HOA_DON_H

#include <vector>
#include <string>
#include "mathang.h"
#include "hoadon.h"
#include "AVLTree.h"

void lapHoaDon(AVLTree<std::string, MatHang>& kho, std::vector<HoaDon>& danhSachHoaDon);
bool kiemTraHetHang(const MatHang& mh, int soLuongMua);
void truTonTuDong(MatHang& mh, int soLuongMua);
void hienThiDanhSachHoaDon(const std::vector<HoaDon>& danhSachHoaDon);

#endif
```

Listing 3.5: Khai báo nguyên mẫu lập hóa đơn tích hợp Cây AVL

Phân hệ Thống kê Doanh thu (`thongkedoanhthu.h`)

```
#ifndef THONGKEDOANHTHU_H
#define THONGKEDOANHTHU_H

#include <vector>
#include "hoadon.h"
#include "ngay.h"

void thongKeDoanhThuTheoNgay(const std::vector<HoaDon>& danhSachHoaDon);
void thongKeDoanhThuTheoThang(const std::vector<HoaDon>& danhSachHoaDon);
void menuThongKeDoanhThu(const std::vector<HoaDon>& danhSachHoaDon);

#endif
```

Listing 3.6: Khai báo nguyên mẫu hàm thống kê doanh thu

Phân hệ Thống kê Top Mặt hàng Bán chạy (`thongketop.h`)

```
struct ThongKeMatHang {
    string maHang;
```

```
    int tongSoLuongBan;
    long long tongDoanhThuMatHang;
};

static bool soSanhBanChay(const ThongKeMatHang& a, const ThongKeMatHang& b) {
    if (a.tongSoLuongBan != b.tongSoLuongBan) {
        return a.tongSoLuongBan > b.tongSoLuongBan;
    }
    return a.tongDoanhThuMatHang > b.tongDoanhThuMatHang;
}

inline void thongKeTopMatHangBanChay(const vector<HoaDon>& danhSachHoaDon, const KhoHang& kho
    ) {
    if (danhSachHoaDon.empty()) {
        cout << "\n[Thong bao] He thong chua co du lieu hoa don nao de thong ke!\n";
        return;
    }

    map<string, ThongKeMatHang> mapThongKe;
    for (size_t i = 0; i < danhSachHoaDon.size(); ++i) {
        const HoaDon& hd = danhSachHoaDon[i];
        for (int j = 0; j < hd.soLuongChiTiet; ++j) {
            const ChiTietHoaDon& ct = hd.danhSachChiTiet[j];
            string ma(ct.maHang);
            mapThongKe[ma].maHang = ma;
            mapThongKe[ma].tongSoLuongBan += ct.soLuongMua;
            mapThongKe[ma].tongDoanhThuMatHang += (long long)ct.soLuongMua * ct.donGia;
        }
    }

    vector<ThongKeMatHang> danhSachThongKe;
    for (map<string, ThongKeMatHang>::const_iterator it = mapThongKe.begin(); it !=
        mapThongKe.end(); ++it) {
        danhSachThongKe.push_back(it->second);
    }
    sort(danhSachThongKe.begin(), danhSachThongKe.end(), soSanhBanChay);
    // In bang Top N mat hang ban chay va tra cuu ten hang qua kho.search(tk.maHang)
}
```

Listing 3.7: Mã nguồn chi tiết phân hệ thống kê top mặt hàng bán chạy

Chương trình Kiểm thử Hiệu năng Benchmark (`Chuongtrinhsinhvadodulieu.h`)

```
inline void chayToanBoBenchmark() {
    std::cout << "\n================================================\n";
    std::cout << "   CHUONG TRINH BENCHMARK HIEU NANG AVL\n";
    std::cout << "================================================\n";

    thucHienDoBenchmark(1000);
    thucHienDoBenchmark(10000);
    thucHienDoBenchmark(100000);
```

```
9
10      std::cout << "\n================================================\n";
11      std::cout << "Kiem thu hieu nang hoan tat thanh cong!\n";
12  }
```

Listing 3.8: Chương trình sinh dữ liệu và đo đạc benchmark thực tế

3.2.4 Giải thích Giải thuật và Thiết kế Kỹ thuật

- **Tương tác Cây AVL tối ưu $O(\log n)$:** Module lập hóa đơn sử dụng tham chiếu đến `AVLTree<string, MatHang>& kho`, cho phép gọi phương thức `search()` để tra cứu thông tin sản phẩm và cập nhật trực tiếp lượng tồn kho trên cây với độ phức tạp đạt $O(\log n)$.
- **Quản lý danh sách chi tiết an toàn:** Giới hạn mảng tĩnh tối đa 20 phần tử cho mỗi hóa đơn giúp kiểm soát tốt bộ nhớ, kết hợp constructor khởi tạo an toàn bằng `std::memset` và kiểu dữ liệu `long long` cho đơn giá và tổng tiền nhằm tránh triệt để hiện tượng tràn số tài chính.
- **Gom nhóm và Sắp xếp hiệu quả:** Thuật toán thống kê top sử dụng `std::map<string, ThongKeMatHang>` để tự động cộng dồn số lượng và doanh thu theo từng mã hàng xuyên suốt các hóa đơn, sau đó chuyển sang `std::vector` để sắp xếp nhanh chóng thông qua hàm `std::sort`.

3.3 Thiết kế và Cài đặt Module Lưu trữ File Nhị phân và Giao diện Console

3.3.1 Mục tiêu và ranh giới lưu trữ

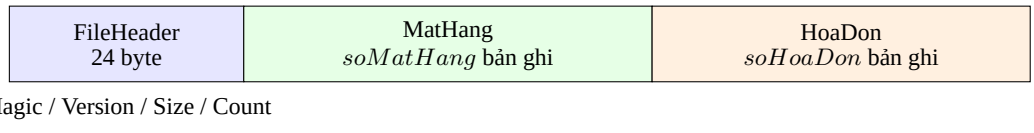
Module FileIO cung cấp lớp biên giữa dữ liệu đang nằm trong RAM và dữ liệu bền vững trên đĩa. Module không phụ thuộc trực tiếp vào chi tiết nút của cây AVL; hàm lưu lấy danh sách đã duyệt theo thứ tự thông qua `inorder()`, còn hàm nạp nhận lại các mặt hàng rồi gọi `clear()` và `insert()` trên cấu trúc kho. Cách thiết kế này giúp tách định dạng file khỏi cấu trúc lưu trữ đang dùng trong RAM.

3.3.2 Đặc tả bố cục file

Dữ liệu kho hàng và danh sách hóa đơn được lưu theo thứ tự tuần tự:

$$[\texttt{FileHeader}] \parallel [\texttt{MatHang} \times soMatHang] \parallel [\texttt{HoaDon} \times soHoaDon].$$

Header gồm 24 byte theo định nghĩa hiện tại: 4 byte mã nhận diện `SMK1`, sau đó là các trường số nguyên 32-bit gồm phiên bản, kích thước bản ghi mặt hàng, kích thước bản ghi hóa đơn, số mặt hàng và số hóa đơn. Kích thước bản ghi được ghi trong header để phát hiện file được tạo bởi bản build có cấu trúc dữ liệu không tương thích.



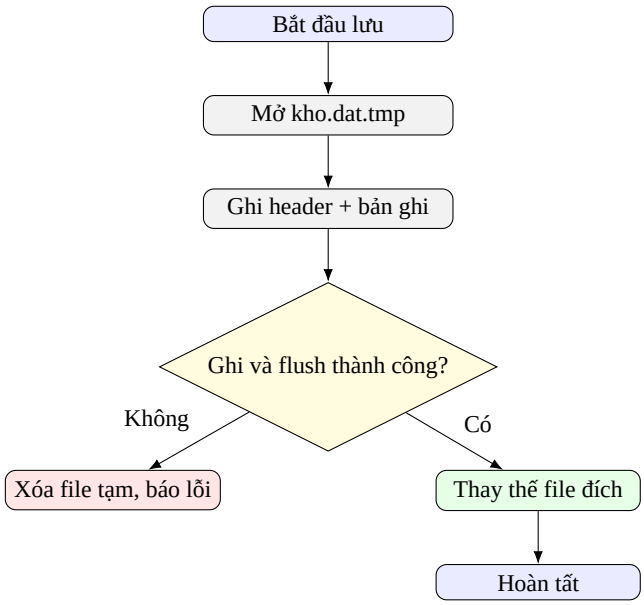
Hình 3.3: Bố cục tuần tự của file kho.dat

Bảng 3.3: Các trường trong FileHeader

Trường	Kiểu	Mục đích
`magic[4]`	4 byte	Nhận diện định dạng file là SMK1.
`version`	uint32_t	Phiên bản schema, hiện tại bằng 1.
`kichThuocMH`	uint32_t	Kích thước bản ghi MatHang của bản build.
`kichThuocHD`	uint32_t	Kích thước bản ghi HoaDon của bản build.
`soMatHang`	uint32_t	Số bản ghi mặt hàng tiếp theo header.
`soHoaDon`	uint32_t	Số bản ghi hóa đơn sau vùng mặt hàng.

3.3.3 Ghi dữ liệu và thay thế file

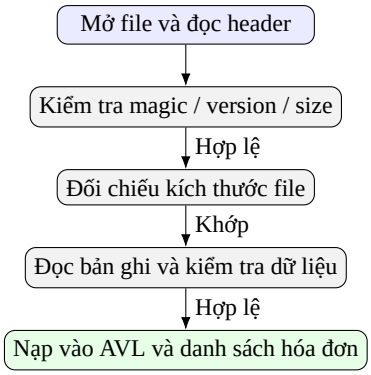
Quy trình ghi gồm các bước: tạo file tạm có hậu tố `.tmp`; ghi header; ghi lần lượt các mặt hàng; ghi các hóa đơn; gọi `fflush`; đóng file; sau đó thay thế file đích bằng file tạm. Nếu lỗi xảy ra trong lúc ghi, file tạm được xóa và hàm trả về thông báo lỗi. Trên hệ điều hành POSIX, thao tác `rename` có thể thay thế đích trên cùng hệ thống file; trên Windows, mã nguồn dùng API thay thế file tương ứng. Cơ chế này giảm nguy cơ để lại file đích chỉ được ghi một nửa, nhưng không thay thế cho sao lưu định kỳ hoặc kiểm tra lỗi thiết bị lưu trữ.



Hình 3.4: Quy trình ghi file qua file tạm

3.3.4 Kiểm tra khi đọc file

Đọc file không chỉ là gọi `fread` theo số lượng được khai báo. Module xác minh lần lượt: file tồn tại; file đủ dài để chứa header; magic khớp; phiên bản được hỗ trợ; kích thước bản ghi tương thích; kích thước thực tế của file bằng tổng kích thước header và các vùng bản ghi được mô tả. Sau khi đọc, các chuỗi được ép kết thúc bằng ký tự null và dữ liệu mặt hàng được kiểm tra các thuộc tính không âm. Khi nạp vào cây, mã hàng trùng bị từ chối.



Hình 3.5: Các bước kiểm tra tính hợp lệ khi nạp file

3.3.5 Xử lý lỗi và bảo đảm trạng thái nhất quán

Hàm `napDuLieu` đọc và kiểm tra dữ liệu vào các vector tạm trước khi xóa nội dung kho hiện tại. Nếu `docFile` thất bại, kho và danh sách hóa đơn đang sử dụng không bị thay đổi. Đây là nguyên tắc quan trọng: chỉ thay thế trạng thái hoạt động sau khi dữ liệu đọc đã vượt qua các bước kiểm tra định dạng. Những lỗi được kiểm tra trong bộ test hiện tại gồm file không tồn tại, file bị cắt cụt, magic sai và dữ liệu mặt hàng không hợp lệ. Khi mở rộng hệ thống, nên bổ sung kiểm tra ngày tháng, mã hóa đơn trùng, tổng tiền và tham chiếu chi tiết hóa đơn đến mã mặt hàng.

3.3.6 Mẫu mã kiểm tra header và độ dài file

```
if (std::memcmp(h.magic, MAGIC, 4) != 0) {
    loi = "Sai dinh dang (magic khong khop)";
    return false;
}
if (h.version != VERSION) {
    loi = "Phien ban file khong ho tro";
    return false;
}
uint64_t expected = sizeof(FileHeader)
    + uint64_t(h.soMatHang) * sizeof(MatHang)
    + uint64_t(h.soHoaDon) * sizeof(HoaDon);
if (expected != uint64_t(size)) {
    loi = "File hong: kich thuoc khong khop";
    return false;
}
```

Listing 3.9: Minh họa các điều kiện kiểm tra cấu trúc file

3.3.7 Giao diện Menu Console

Giao diện console được chia thành menu chính và các menu con: quản lý kho; hóa đơn/doanh thu; lưu trữ; chạy benchmark; xem nhanh cảnh báo. Menu chính hiển thị số lượng mặt hàng, chiều cao cây và số hóa đơn hiện có để người dùng có phản hồi về trạng thái chương trình. Mỗi menu con nhận lựa chọn từ hàm đọc số nguyên có kiểm tra miền giá trị.

Nếu người dùng nhập ký tự thay vì số, luồng `std::cin` chuyển sang trạng thái lỗi. Chương trình gọi `cin.clear()` để khôi phục trạng thái và `cin.ignore()` để bỏ phần còn lại của dòng. Nếu không xử lý hai bước này, các lần đọc sau có thể tiếp tục thất bại trên cùng dữ liệu lỗi, tạo cảm giác chương trình bị treo.

3.3.8 Xử lý ngoại lệ và vòng đời bộ nhớ

Khối `main()` được đặt trong `try/catch` để chuyển các ngoại lệ chưa xử lý thành thông báo có kiểm soát. Cây AVL có destructor giải phóng các nút khi đối tượng rời khỏi phạm vi. Việc thu hồi bộ nhớ theo RAII giảm nguy cơ rò rỉ khi kết thúc chương trình bình thường; tuy nhiên, khẳng định không có rò rỉ trong mọi đường chạy cần được kiểm tra riêng bằng công cụ chuyên dụng như AddressSanitizer hoặc Valgrind trên môi trường phù hợp. Báo cáo không xem sự tồn tại của destructor là bằng chứng thay thế cho kiểm thử bộ nhớ.

3.3.9 Kiểm thử FileIO và các giới hạn

Bộ kiểm thử FileIO hiện thực các ca: ghi rồi đọc lại dữ liệu; xử lý file rỗng hợp lệ; từ chối file không tồn tại; từ chối file bị cắt; từ chối magic sai; từ chối bản ghi mặt hàng có giá trị vô lý. Phần kiểm thử tích hợp kho còn xác nhận có thể lưu danh sách mặt hàng và hóa đơn rồi nạp lại vào một cây AVL khác.

Định dạng hiện tại ghi trực tiếp biểu diễn nhị phân của struct, vì vậy kích thước và cách bố trí struct phụ thuộc compiler, ABI và cách căn chỉnh bộ nhớ. Header chứa kích thước struct giúp phát hiện một số trường hợp không tương thích nhưng không tạo ra định dạng di động tuyệt đối giữa mọi nền tảng. Một phiên bản nâng cấp nên ghi từng trường theo thứ tự và quy ước byte cụ thể, có checksum, giới hạn số bản ghi, và cơ chế chuyển đổi schema khi đổi phiên bản.

Bảng 3.4: Các ca kiểm thử FileIO đã chạy

Mã	Tình huống	Kết quả mong đợi	Kết quả chạy
F01	Ghi và đọc lại	Dữ liệu mặt hàng/hóa đơn giữ nguyên.	Đạt
F02	File rỗng	Đọc thành công với hai danh sách rỗng.	Đạt
F03	File không tồn tại	Trả lỗi, không làm thay đổi kho.	Đạt
F04	File bị cắt	Từ chối do kích thước không khớp.	Đạt
F05	Magic sai	Từ chối do định dạng không khớp.	Đạt
F06	Dữ liệu âm/vô lý	Từ chối bản ghi.	Đạt

3.3.10 Hướng dẫn thao tác demo

Để minh họa module, người trình bày chạy `./main_app`, kiểm tra thông báo nạp CSV ban đầu, mở menu lưu trữ và thực hiện lưu dữ liệu. Sau đó có thể thoát và chạy lại chương trình để quan sát cơ chế phục hồi từ file. Một ca kiểm thử lỗi nên chạy độc lập trong bộ test FileIO thay vì cố làm hỏng file dữ liệu đang dùng cho demo. Khi trình bày, cần phân biệt rõ dữ liệu mẫu CSV dùng khởi tạo lần đầu với file nhị phân dùng lưu trạng thái sau giao dịch. Khi khởi động, `main` phân biệt ba tình huống: (1) `kho.dat` đọc được thì khôi phục dữ liệu; (2) `kho.dat` chưa tồn tại thì nạp dữ liệu mẫu CSV; (3) `kho.dat` tồn tại nhưng không đọc được thì báo lỗi kèm lý do và **không tự ghi đè**. Ở tình huống (3), người dùng chọn thoát để giữ nguyên file, hoặc chọn sao lưu file lỗi thành `kho.dat.hongN` (không bao giờ ghi đè bản sao lưu cũ) rồi mới dùng dữ liệu mẫu; nếu sao lưu thất bại thì chương trình dừng. Hàm kiểm tra sự tồn tại chỉ coi là ”chưa có file” khi hệ điều hành báo `ENOENT`; mọi lỗi khác (không có quyền đọc, đường dẫn là thư mục) đều đi vào nhánh báo lỗi. Kịch bản này được kiểm thử tự động bằng `tests/test_khoi_dong.sh` trong thư mục tạm, không đụng tới `kho.dat` thật.

3.3.11 Đầu ra kiểm thử và cách đọc thông báo

Trong bản ghi kiểm thử, thông báo bắt đầu bằng `[OK]` biểu thị ca kiểm thử đã đạt. Một số dòng có tiền tố `[Lỗi]` được tạo có chủ đích để xác nhận chương trình từ chối dữ liệu không hợp lệ; đó không phải lỗi làm hỏng tiến trình test. Khi báo cáo kết quả, cần phân biệt lỗi nghiệp vụ được kiểm tra với lỗi assertion thực sự.

CHƯƠNG 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG

4.1 Mục tiêu thực nghiệm

Chương này so sánh chi phí tìm kiếm chính xác theo mã hàng của cây AVL với vector đã sắp xếp sử dụng `std::lower_bound`. Đây là baseline công bằng hơn so với duyệt tuyến tính: cả hai phương pháp đều khai thác thứ tự khóa và có số phép so sánh tăng theo logarithm. Ngoài tìm kiếm, benchmark đo riêng 100 lần chèn và 100 lần xóa để thể hiện chi phí cập nhật của hai cấu trúc.

4.2 Môi trường và phương pháp đo

Lần chạy được thực hiện ngày 31/10/2026 trong môi trường Linux x86_64, nhân 6.18.57, GCC 14.2.0, 3 CPU logic được cấp cho container; cờ biên dịch là `-std=c++11 -O2 -Wall -Wextra`. Đây là môi trường ảo hóa/container, không phải phép đo trên máy demo của nhóm. Vì vậy, thời gian tuyệt đối có thể khác trên máy cá nhân hoặc máy giảng viên.

Chương trình tạo lần lượt 1.000, 10.000, 50.000 và 100.000 mặt hàng. Cả AVL và vector được nạp cùng tập mã khóa. Vector luôn được duy trì theo thứ tự tăng dần; tìm kiếm dùng `lower_bound`. Một tập 1.000 khóa đã tồn tại, được sinh bằng `std::mt19937` với seed 2026, được dùng chung cho hai cấu trúc. Sau một lượt làm nóng, mỗi phương pháp được đo 9 lần bằng `steady_clock`; báo cáo lấy trung vị thời gian tổng của 1.000 truy vấn. Chèn và xóa cũng được đo riêng: mỗi lượt gồm 100 thao tác trên bản sao dữ liệu đã chuẩn bị trước, đo 9 lần và lấy trung vị. Thời gian khởi tạo/copy cấu trúc không được tính vào khoảng đo cập nhật.

4.3 Kết quả đo tìm kiếm

Bảng 4.1 trình bày trung vị tổng thời gian của 1.000 truy vấn. Tất cả truy vấn đều tìm thấy khóa tương ứng ở cả hai cấu trúc.

Bảng 4.1: Trung vị thời gian tìm kiếm 1.000 khóa (9 lần đo)

Số mặt hàng	Chiều cao AVL	AVL (μs)	Vector + lower_bound (μs)	Vector/AVL
1.000	10	100,089	64,496	0,64 lần
10.000	14	192,745	122,081	0,63 lần
50.000	16	284,702	187,238	0,66 lần
100.000	17	389,176	216,581	0,56 lần

4.4 Kết quả đo chèn và xóa

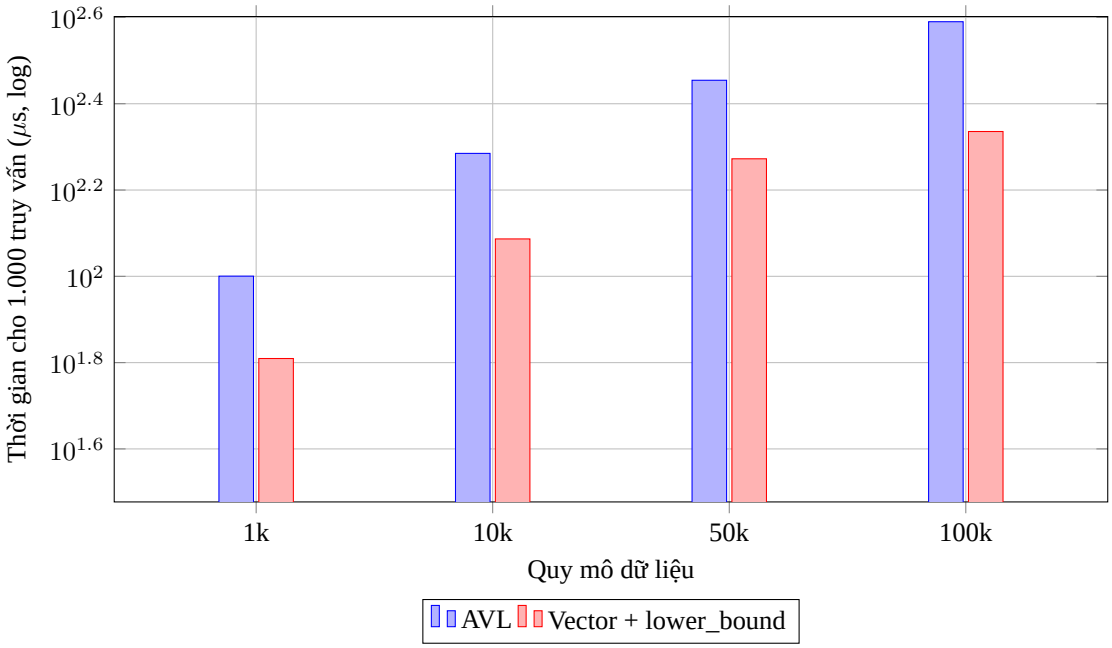
Bảng 4.2 ghi nhận tổng thời gian của 100 thao tác chèn hoặc xóa. Các thao tác trên vector dùng tìm vị trí bằng `lower_bound`, sau đó `insert`/`erase` phải dịch chuyển phần tử để giữ thứ tự.

Bảng 4.2: Trung vị thời gian của 100 thao tác cập nhật (9 lần đo)

Số mặt hàng	AVL chèn (μs)	Vector chèn (μs)	AVL xóa (μs)	Vector xóa (μs)
1.000	10,816	323,700	12,438	188,830
10.000	14,231	4.321,075	32,068	2.181,564
50.000	21,362	21.464,294	129,231	10.705,357
100.000	23,595	40.817,120	177,273	21.948,953

4.5 Trực quan hóa kết quả tìm kiếm

Biểu đồ 4.1 so sánh hai phương pháp tìm kiếm bằng thang logarit. Vector đã sắp xếp nhanh hơn trong lần chạy này, dù cả hai phương pháp đều có độ phức tạp tìm kiếm logarithm theo số phần tử.



Hình 4.1: So sánh thời gian tìm kiếm bằng AVL và vector đã sắp xếp

4.6 Phân tích kết quả và liên hệ độ phức tạp

Ở quy mô 100.000 mặt hàng, AVL có chiều cao 17 và cần 389,176 μs cho 1.000 truy vấn; vector đã sắp xếp với `lower_bound` cần 216,581 μs. Trong lần đo này, vector nhanh hơn khoảng 1,79 lần. Kết quả không mâu thuẫn với độ phức tạp lý thuyết: tìm kiếm trên AVL là $O(\log n)$ theo chiều cao cây, còn tìm kiếm nhị phân trên vector đã sắp xếp cũng là $O(\log n)$.

Chênh lệch thời gian thực tế chịu ảnh hưởng của hằng số chi phí và kiến trúc bộ nhớ. Vector lưu phần tử liên tục nên tận dụng locality/cache tốt hơn; thao tác nhị phân thường truy cập ít vùng nhớ và không cần lần theo con trỏ node. AVL phải đi qua các node được cấp phát rời rạc, nên pointer chasing và cache miss có thể làm tăng độ trễ dù chiều cao cây nhỏ. Do vậy, không thể kết luận rằng AVL luôn nhanh hơn vector trong mọi tác vụ chỉ dựa vào ký hiệu $O(\log n)$.

Ở chiều ngược lại, cập nhật trên vector phải dịch chuyển các phần tử để duy trì thứ tự. Tại 100.000 mặt hàng, 100 lần chèn mất 40.817,120 μs trên vector so với 23,595 μs trên AVL; 100 lần xóa mất 21.948,953 μs trên vector so với 177,273 μs trên AVL. Số đo cho thấy cấu trúc phù hợp phụ thuộc vào tỷ lệ đọc/ghi: vector có lợi thế

tra cứu trong bộ nhớ liên tục, còn cây AVL phù hợp hơn khi cần cập nhật thường xuyên và duy trì thứ tự khóa.

4.7 Giới hạn của phép đo

Kết quả là trung vị của 9 lần đo trong một môi trường container, với một bộ dữ liệu và các truy vấn thành công. Benchmark chưa đo truy vấn không tìm thấy, RAM, cấp phát bộ nhớ, lưu file, tải đồng thời, hoặc độ trễ từ thao tác người dùng. Mức nhiễu do lịch CPU, cache, compiler và cấu hình máy vẫn có thể ảnh hưởng. Vì vậy, số liệu chỉ mô tả lần chạy kèm gói bàn giao, không phải cam kết hiệu năng trên máy khác.

4.8 Trích xuất đầu ra benchmark

Đầu ra stdout thực tế của lần chạy được lưu tại `benchmark_console.txt`. Có thể đối chiếu trực tiếp các giá trị trong bảng với tệp log này.

```
1
2  ================================================
3     CHUONG TRINH BENCHMARK HIEU NANG AVL
4  ================================================
5
6  ================================================
7    QUY MO KIEM THU: 1000 MAT HANG
8  ================================================
9   - Chieu cao cay AVL (1000 nut): 10
10  - So truy van khop: 1000/1000 (1000 truy van; 9 lan do)
11  - Trung vi AVL:     100.089 microseconds / 1000 truy van
12  - Trung vi Vector (sorted + lower_bound): 64.496 microseconds / 1000 truy van
13  ==> Vector/AVL (tim kiem): 0.64 lan
14  - Them 100 khoa | AVL: 10.816 us | Vector sorted: 323.700 us
15  - Xoa  100 khoa | AVL: 12.438 us | Vector sorted: 188.830 us
16
17 ================================================
18   QUY MO KIEM THU: 10000 MAT HANG
19 ================================================
20  - Chieu cao cay AVL (10000 nut): 14
21  - So truy van khop: 1000/1000 (1000 truy van; 9 lan do)
22  - Trung vi AVL:     192.745 microseconds / 1000 truy van
23  - Trung vi Vector (sorted + lower_bound): 122.081 microseconds / 1000 truy van
24  ==> Vector/AVL (tim kiem): 0.63 lan
25  - Them 100 khoa | AVL: 14.231 us | Vector sorted: 4321.075 us
26  - Xoa  100 khoa | AVL: 32.068 us | Vector sorted: 2181.564 us
27
28 ================================================
29   QUY MO KIEM THU: 50000 MAT HANG
30 ================================================
31  - Chieu cao cay AVL (50000 nut): 16
32  - So truy van khop: 1000/1000 (1000 truy van; 9 lan do)
33  - Trung vi AVL:     284.702 microseconds / 1000 truy van
34  - Trung vi Vector (sorted + lower_bound): 187.238 microseconds / 1000 truy van
35  ==> Vector/AVL (tim kiem): 0.66 lan
```

```
 - Them 100 khoa | AVL: 21.362 us | Vector sorted: 21464.294 us
 - Xoa  100 khoa | AVL: 129.231 us | Vector sorted: 10705.357 us

================================================
  QUY MO KIEM THU: 100000 MAT HANG
================================================
 - Chieu cao cay AVL (100000 nut): 17
```

Listing 4.1: Trích đoạn stdout của lần benchmark đã chạy

4.9 Kết luận chương

Trong lần thực nghiệm này, AVL và vector đã sắp xếp có hiệu năng tìm kiếm cùng mức độ tăng trưởng logarithm; vector nhanh hơn về tra cứu nhờ tính cục bộ bộ nhớ, trong khi AVL có lợi thế rõ rệt ở các phép cập nhật được đo. AVL vẫn là cấu trúc lõi phù hợp với yêu cầu quản lý kho có thêm/xóa mặt hàng, nhưng quyết định cuối cùng cần xét đồng thời tỷ lệ đọc/ghi, bộ nhớ và yêu cầu nghiệp vụ.

KẾT LUẬN

Kết quả đạt được

Nhóm đã xây dựng bản tích hợp hệ thống quản lý hàng tồn kho và doanh thu siêu thị sử dụng cây AVL tự cài đặt làm cấu trúc dữ liệu lõi. Trên phiên bản mã nguồn được ghép từ năm gói thành viên và cập nhật trong đợt này, các kết quả có thể kiểm chứng gồm:

- Cài đặt `AVLTree<K,V>` với thêm, xóa, tìm kiếm, duyệt, range search, kích thước/chiều cao và kiểm tra rỗng. Bộ kiểm thử cập nhật đạt 35/35 ca, trong đó có kiểm tra tính `const` của kết quả tìm kiếm và move constructor/assignment.
- Module kho hỗ trợ CRUD, nhập/xuất, chặn xuất quá tồn, kiểm tra tràn số khi nhập, kiểm tra ngày theo lịch và nạp CSV có xác thực từng dòng. Các kịch bản tích hợp kho hiện có chạy đạt.
- Module hóa đơn chỉ trừ tồn sau khi người dùng xác nhận toàn bộ giao dịch; EOF hoặc từ chối xác nhận hủy hóa đơn và giữ nguyên tồn kho. Mã hóa đơn trùng, số lượng không hợp lệ, dòng hàng lặp và tổng tiền vượt giới hạn được kiểm tra. Bộ test hóa đơn–doanh thu đạt 55/55 ca.
- Module FileIO kiểm tra magic/version, kích thước, mã hàng trùng, trường dữ liệu mặt hàng và chi tiết hóa đơn; khi ghi, vùng đệm của struct được khởi tạo trước khi tuần tự hóa. Các ca FileIO được mở rộng và chạy đạt.
- Makefile đã theo dõi phụ thuộc header qua `-MMD -MP`, có target `asan`; bộ test khởi động đạt 12/12 ca và xác nhận file kho hỏng không bị tự động ghi đè.

Kết quả benchmark

Lần benchmark cập nhật ngày 31/10/2026 chạy với GCC 14.2.0 trên Linux x86_64, container được cấp 3 CPU logic. Mỗi cấu trúc thực hiện cùng 1.000 truy vấn; chạy làm nóng trước, đo 9 lần và lấy trung vị. Ở 100.000 mặt hàng, AVL có chiều cao 17 và mất 389,176 μs; vector đã sắp xếp với `lower_bound` mất 216,581 μs cho 1.000 truy vấn. Như vậy vector nhanh hơn khoảng 1,79 lần trong phép đo tìm kiếm này, phù hợp với lợi thế locality/cache của vùng nhớ liên tục; cả hai phương pháp đều có chi phí tìm kiếm logarithm.

Với 100 thao tác cập nhật tại 100.000 mặt hàng, AVL mất 23,595 μs cho chèn và 177,273 μs cho xóa; vector đã sắp xếp lần lượt mất 40.817,120 μs và 21.948,953 μs do phải dịch chuyển phần tử. Kết quả cho thấy không thể kết luận AVL luôn nhanh hơn trong mọi tác vụ; lợi thế của mỗi cấu trúc phụ thuộc vào tỷ lệ tra cứu/cập nhật. Các bảng và phương pháp đo chi tiết nằm ở Chương 4, dữ liệu stdout được lưu trong `benchmark_console.txt`.

Hạn chế

- Ứng dụng vẫn là giao diện console, chưa có xác thực, phân quyền hoặc giao diện đồ họa.
- Dữ liệu kho được lưu bằng định dạng nhị phân gắn với cách bố trí struct của bản build; chưa phải định dạng trao đổi độc lập nền tảng.

- Tra cứu theo khoảng giá vẫn phải duyệt dữ liệu vì cây AVL hiện được khóa theo mã hàng; chưa có chỉ mục phụ theo giá.
- Mỗi hóa đơn giới hạn tối đa 20 dòng chi tiết.
- Benchmark được thực hiện trong một môi trường container và với một bộ truy vấn; chưa đánh giá RAM, truy vấn thất bại, tải đồng thời hay thời gian lưu file.

Công việc cần xác nhận trước khi nộp

Các gói file có thể chứng minh mã nguồn, nội dung LaTeX, slide, kịch bản demo và kết quả build/test trong môi trường hiện tại. Tuy nhiên, chúng không cung cấp lịch sử commit, dữ liệu Jira, URL/trạng thái pull request, email gửi giảng viên, biên nhận nộp bài hoặc biên bản tập dượt/đọc chéo. Vì vậy, nhóm cần thực hiện và lưu những bằng chứng này trực tiếp trước khi đánh dấu các task quy trình là hoàn tất. Tỷ lệ đóng góp của từng người phải được tính từ commit và công việc thực tế, không tự đặt mặc định.

Hướng phát triển

Các hướng mở rộng gồm xây dựng chỉ mục phụ theo đơn giá; chuyển sang cơ sở dữ liệu có giao dịch; chuẩn hóa định dạng lưu file độc lập ABI và có checksum; thêm kiểm thử bộ nhớ; bổ sung tìm kiếm theo tên/nhóm hàng; xây dựng giao diện đồ họa hoặc web; và mở rộng benchmark sang truy vấn thất bại, bộ nhớ, tải đồng thời cũng như các phân phối dữ liệu khác.

TÀI LIỆU THAM KHẢO

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[2] G. M. Adelson-Velsky, E. M. Landis, “An algorithm for the organization of information”, *Doklady Akademii Nauk SSSR*, 146, 1962.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998.

[4] B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013.

PHỤ LỤC: PHÂN CÔNG CÔNG VIỆC VÀ TÌNH TRẠNG XÁC MINH

Bảng dưới đây mô tả phạm vi công việc được phân công trong kế hoạch và các tệp bàn giao có trong gói hiện tại. Tỷ lệ đóng góp thực tế không được suy ra từ số lượng file hoặc số task được giao. Cần đối chiếu lịch sử commit, nội dung pull request và trạng thái Jira trước khi ghi nhận phần trăm đóng góp chính thức.

Bảng 4.3: Phạm vi phân công và bằng chứng trong gói bàn giao

Thành viên	Nhánh dự kiến	Phạm vi chính	Task	Tỷ lệ thực tế
TV1 – Đoàn Quốc Nhật	`main-integration`	Tích hợp, Makefile, chương trình chính, khung và kết luận báo cáo.	T01–T15	Chưa xác minh
TV2 – Huỳnh Long Nhật	`avl-tree`	Cây AVL, bộ test cấu trúc dữ liệu, Chương 2.	T16–T25	Chưa xác minh
TV3 – Trần Văn Tài	`module-kho`	Module kho, dữ liệu mẫu, Chương 1 và phần kho Chương 3.	T26–T34	Chưa xác minh
TV4 – Hoàng Xuân Nhuật	`module-doanh-thu`	Hóa đơn, doanh thu, benchmark, Chương 4.	T35–T43	Chưa xác minh
TV5 – Nguyễn Duy Khang	`fileio-ui`	FileIO, giao diện console, slide và kịch bản demo.	T44–T52	Chưa xác minh
Cả nhóm	`main`	Đọc chéo báo cáo và tập dượt demo.	T52–T53	Cần biên bản

Quy tắc xác nhận phần trăm đóng góp

Khi hoàn thiện phụ lục cuối cùng, nhóm nên thống nhất một khoảng thời gian thống kê; xuất danh sách commit theo từng tác giả; loại commit merge, commit bot và commit trùng lặp; đối chiếu thay đổi thực chất của từng pull request; sau đó so sánh với task Jira đã được nghiệm thu. Số dòng thêm/xóa không nên là tiêu chí duy nhất vì một thay đổi thiết kế hoặc sửa lỗi nhỏ có thể có giá trị khác với việc sinh mã tự động.

Danh sách minh chứng quy trình cần lưu

- Đường dẫn repository và ảnh/chụp xuất lịch sử commit theo thành viên.
- Link pull request, người review, ngày merge và trạng thái cuối của từng nhánh.
- Bản xuất Jira thể hiện assignee, deadline, trạng thái và người nghiệm thu cho các task.
- Email hoặc xác nhận chia sẻ bản LaTeX cho giảng viên (T04).
- Phiếu/biên bản đọc chéo báo cáo của các thành viên (T53).
- Biên bản tập dượt demo hoặc checklist chạy thử có người tham gia (T52).

Không có các minh chứng trên trong các tệp được cung cấp ở lần rà soát này. Vì vậy, phụ lục sử dụng nhãn “chưa xác minh” thay vì ghi 20% đồng đều cho từng thành viên.

PHỤ LỤC: HƯỚNG DẪN BIÊN DỊCH, KIỂM THỬ VÀ NGHIỆM THU

A. Cấu trúc thư mục bàn giao

Dự án được chia thành các thư mục theo trách nhiệm kỹ thuật. Thư mục `include/` chứa khai báo cấu trúc và giao diện module; `src/` chứa các định nghĩa hàm và điểm vào chương trình; `tests/` chứa bộ kiểm thử; `data/` chứa dữ liệu CSV mẫu; `docs/baocao/` chứa các tệp LaTeX; `docs/demo_slide/` chứa slide và kịch bản demo. Cấu trúc này giúp phân biệt mã chạy, mã kiểm thử và tài liệu.

Bảng 4.4: Các tệp nguồn chính trong dự án tích hợp

Tệp	Vai trò	Ghi chú
`include/AVLTree.h`, `AVLTree.tpp`	Cấu trúc AVL	Giao diện và phần cài đặt template.
`include/mathang.h`	Mô hình mặt hàng	Kiểu dữ liệu dùng chung.
`src/quan_ly_kho.cpp`	Nghiệp vụ kho	CRUD, nhập/xuất, cảnh báo, nạp CSV.
`src/lap_hoa_don.cpp`	Hóa đơn	Kiểm tra tồn và trừ tồn.
`src/thongkedoanhthu.cpp`	Thống kê	Doanh thu ngày/tháng.
`include/FileIO.h`	Lưu trữ	Ghi và đọc file nhị phân.
`src/main.cpp`	Giao diện	Điều phối menu và module.
`src/benchmark.cpp`	Benchmark	Chạy phép so sánh AVL/vector độc lập.

B. Các lệnh biên dịch và kiểm thử

Các lệnh dưới đây được chạy từ thư mục gốc repository. Cần có trình biên dịch GCC hỗ trợ C++11 và công cụ `make`. Build có thể thực hiện như sau:

```
make clean
make
```

Listing 4.2: Biên dịch toàn bộ chương trình

Lệnh `make` tạo `main_app` và `benchmark_app`. Chương trình chính có thể chạy bằng:

```
./main_app
./benchmark_app
```

Listing 4.3: Chạy menu và chương trình benchmark

Bộ test tích hợp được biên dịch và chạy bằng:

```
make test
```

Listing 4.4: Chạy các nhóm kiểm thử và tích hợp

Khi kiểm tra thủ công, nên bắt đầu ở thư mục gốc để đường dẫn tương đối tới `data/du_lieu_mau.csv` được phân giải đúng. Không nên chạy trực tiếp từ thư mục `src/` vì các đường dẫn tương đối của ứng dụng được thiết kế theo gốc repository.

C. Ma trận độ phức tạp thao tác

Bảng dưới đây tóm tắt độ phức tạp theo cấu trúc hiện tại. Ký hiệu n là số mặt hàng, k là số kết quả trong khoảng khóa, d là số hóa đơn và c là tổng số dòng chi tiết của các hóa đơn được duyệt.

Bảng 4.5: Độ phức tạp của các thao tác chính

Thao tác	**Độ phức tạp**	**Giải thích**
Tìm kiếm theo mã	$O(\log n)$	Đi theo một nhánh cây AVL.
Thêm mặt hàng	$O(\log n)$	Tìm vị trí chèn và cập nhật cân bằng.
Xóa mặt hàng	$O(\log n)$	Tìm nút, xóa và cân bằng đường đi lên gốc.
Duyệt toàn bộ kho	$O(n)$	Mỗi nút được thăm đúng một lần.
Range search theo mã	$O(\log n + k)$	Tìm vùng bắt đầu và duyệt các nút thuộc đoạn.
Lọc theo khoảng giá	$O(n)$	Duyệt toàn bộ vì cây không khóa theo giá.
Lập hóa đơn có r dòng	$O(r \log n)$	Mỗi dòng tìm một mã hàng trong cây; bỏ qua chi phí nhập/xuất console.
Cảnh báo tồn tối thiểu	$O(n)$	Kiểm tra điều kiện trên toàn bộ mặt hàng.
Lưu kho và hóa đơn	$O(n + d + c)$	Ghi toàn bộ bản ghi mặt hàng, hóa đơn và chi tiết.
Nạp file và dựng cây	$O(n \log n + d + c)$	Đọc bản ghi rồi chèn từng mặt hàng vào AVL.

Các độ phức tạp nêu trên là mô tả số thao tác theo kích thước đầu vào, không bao gồm chi phí nhập xuất console hoặc thời gian truy cập thiết bị lưu trữ cụ thể. Khi cần phân tích thực tế, phải đo riêng từng tác vụ và mô tả kích thước đầu vào tương ứng.

D. Bảng kiểm thử nghiệm thu

Bảng 4.6: Checklist các bằng chứng kỹ thuật được ghi nhận

Nhóm	**Bằng chứng trong lần kiểm tra**	**Kết quả**
Build	`make clean` và `make` với GCC 14.2.0, C++11, `-O2`.	Thành công
AVL	12 phép quay, 12 ca xóa, 2 ca const/move, 9 ca range search.	35/35 đạt
FileIO	Ghi/đọc, file rỗng, file thiếu, file cụt, magic/version sai, dữ liệu mặt hàng vô lý, mã trùng, chi tiết hóa đơn không hợp lệ; file 3 byte và 0 byte bị từ chối; nạp file hỏng không làm đổi kho đang có.	13 kiểm tra đạt
Kho	Nạp CSV, CRUD, nhập/xuất, chặn vượt tồn, chặn tràn `INT_MAX` khi nhập kho, ngày theo lịch thực tế (31/02, 31/04, năm nhuận), cảnh báo, range search, lưu/nạp.	Các assertion đạt
Hóa đơn	Lập hóa đơn theo giao dịch; EOF/hủy xác nhận không trừ kho; mã trùng, lặp mã hàng, tổng tiền tràn; thống kê ngày/tháng.	55/55 đạt
Khởi động	`tests/test_khoi_dong.sh`: chưa có `kho.dat`; file 3 byte / 0 byte / bị cắt không bị ghi đè; sao lưu `kho.dat.hongN`; khôi phục file hợp lệ.	12/12 đạt
Benchmark	4 quy mô; 1.000 truy vấn; 9 lần đo; vector sorted + lower_bound và 100 chèn/xóa.	Có bảng, biểu đồ và stdout
Báo cáo	Hai lượt XeLaTeX, rà soát PDF và số trang sau khi ghép phụ lục.	42 trang A4
Sanitizer	`make asan` với AddressSanitizer và UndefinedBehaviorSanitizer; tất cả test chạy qua.	Thành công

Các trạng thái trên chỉ phản ánh các lệnh đã chạy trong môi trường kiểm tra của lần bàn giao này. Chúng không chứng minh rằng pull request đã được merge trên GitHub, task đã được cập nhật trên Jira hoặc giảng viên đã nhận được bản LaTeX.

E. Checklist thao tác demo

Trình tự demo nên được tiến hành trên một bản build sạch để tránh sử dụng file nhị phân từ lần chạy cũ:

1. Chạy `make clean` rồi `make`; xác nhận hai executable được tạo.
2. Chạy `./main_app`; quan sát thông báo nạp 50 mặt hàng CSV nếu chưa tồn tại `kho.dat`.
3. Mở phân hệ kho; duyệt danh sách theo mã; tìm một mã có thật và một mã không có.
4. Thêm mã mới; thử thêm lại cùng mã để trình bày cơ chế từ chối trùng.
5. Thử xuất vượt tồn và xác nhận số lượng tồn không bị thay đổi; tiếp đó xuất một lượng hợp lệ.
6. Mở phân hệ hóa đơn; lập hóa đơn nhỏ với một hoặc hai mặt hàng; đối chiếu tổng tiền và tồn sau bán.
7. Lưu dữ liệu; thoát chương trình; mở lại để xác nhận phục hồi từ file nhị phân.
8. Trình bày cơ chế bảo vệ dữ liệu: chạy `sh tests/test_khoi_dong.sh` để kiểm tra `kho.dat` hỏng không bị ghi đè (script chạy trong thư mục tạm, không làm hỏng file dữ liệu của buổi demo).
9. Chạy `./benchmark_app`; trình bày cách đọc bảng, số truy vấn, trung vị và giới hạn của kết quả.

F. Câu hỏi phản biện nên chuẩn bị

- Vì sao chọn AVL thay cho BST thường? Cần trả lời bằng giới hạn chiều cao và ảnh hưởng của cây suy biến.
- Vì sao range search theo mã là $O(\log n + k)$ nhưng tìm theo khoảng giá là $O(n)$? Vì khóa trong cây là mã hàng, không phải giá bán.
- Vì sao thời gian AVL có thể khác giữa hai máy? Cần đề cập CPU, compiler, cờ tối ưu, tải nền, cache và phương pháp đo.
- Vì sao không ghi 0 microsecond? Vì phép đo có thể nhỏ hơn một đơn vị của kiểu thời gian bị ép kiểu; phiên bản mới đo nanosecond và hiển thị microsecond dạng thập phân.
- Vì sao file nhị phân có thể không đọc được trên bản build khác? Vì ghi trực tiếp struct chịu ảnh hưởng ABI và kích thước trường; header chỉ phát hiện một số trường hợp không tương thích.
- Điều gì xảy ra khi file hỏng? Module đọc vào vùng tạm, kiểm tra header/kích thước/nội dung và chỉ nạp khi hợp lệ. Khi khởi động gặp `kho.dat` hỏng, chương trình báo lỗi và không tự ghi đè; người dùng chọn thoát hoặc sao lưu thành `kho.dat.hongN` rồi dùng dữ liệu mẫu.
- Vì sao phải kiểm tra tràn số trước phép cộng? Vì tràn số nguyên có dấu là hành vi không xác định; so sánh `soLuongNhap > INT_MAX - soLuongTon` không bao giờ thực hiện phép cộng gây tràn.
- Vì sao cần kiểm tra ngày theo lịch? Chỉ kiểm tra ngày trong khoảng 1–31 sẽ nhận 31/02; hàm `ngayHopLe()` dùng số ngày thực của từng tháng và quy tắc năm nhuận.

G. Minh chứng quy trình còn phải bổ sung

Trước khi chốt bản nộp, nhóm cần lưu các bằng chứng không thể kiểm tra từ mã nguồn trong gói: (1) link hoặc ảnh chụp Jira với assignee, trạng thái và deadline; (2) lịch sử commit của từng thành viên và pull request tương ứng; (3) xác nhận chia sẻ bản LaTeX cho giảng viên ở T04; (4) ghi chú đọc chéo báo cáo ở T53; và (5) checklist hoặc biên bản tập dượt demo ở T52. Các nội dung này cần được bổ sung bởi thành viên có quyền truy cập hệ thống thực tế; báo cáo không tự gán trạng thái hoàn tất.

H. Mẫu biên bản đọc chéo và tập dượt demo

Mẫu dưới đây được để trống có chủ đích để các thành viên ghi nhận hoạt động thực tế. Khi hoàn tất, nên điền ngày giờ, người tham gia, lỗi phát hiện và hướng xử lý; không đánh dấu trước những nội dung chưa diễn ra.

Bảng 4.7: Mẫu ghi nhận tập dượt demo (T52)

Thông tin	Nội dung ghi nhận
Ngày, giờ tập dượt	
Thành viên tham gia	
Máy và môi trường chạy	
Phiên bản commit/build	
Lỗi phát hiện	
Người nhận xử lý	
Kết quả chạy lại	
Xác nhận của nhóm	

Bảng 4.8: Mẫu ghi nhận đọc chéo báo cáo (T53)

Người đọc	Chương/phần	Góp ý hoặc lỗi cần sửa	Trạng thái
TV1			
TV2			
TV3			
TV4			
TV5			

Sau khi điền, nhóm lưu biên bản cùng phiên bản báo cáo đã chốt để có thể truy nguyên góp ý. Nếu không phát hiện lỗi, vẫn nên ghi rõ phần đã đọc, ngày đọc và xác nhận của người đọc; để trống không đồng nghĩa với đã review.